NED UNIVERSITY OF ENGINEERING AND TECHNOLOGY

COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

DISASTER RELIEF
COORDINATION SYSTEM

Because Every Second Lost is a Life Lost

7	5	6	4
Disaster Types Managed	Design Patterns Applied	Persistent Data Files	National Languages

Submitted To:
Ms. Murk Marvi
Sir. Ahmed Zaki

Submitted By:
Zainab Kinza Sheikh — CT- 25059
Saita Ahuja — CT- 25065
Aatiqa Batool — CT- 25056
(Section: B)

DISTRIBUTION & CONTRIBUTION TABLE

#	Member Name	Roll No.	Work Distribution / Contribution
01	Saita Ahuja	CT-25065	UML, Code and Report Section 1-7
02	Zainab Kinza Sheikh	CT-25059	UML, Code and Report Section 8-14
03	Aatiqa Batool	CT-25056	UML, Code and Report Section 15-21

TABLE OF CONTENTS

1	**Problem Domain**	5
	1.1 Pakistan's Disaster Crisis: A Nation Without a System	
	1.1.1 The 2010 Super Floods	
	1.1.2 The 2005 Kashmir Earthquake	
	1.1.3 Cyclone Biparjoy 2023	
	1.1.4 2022 Floods: Pakistan's Worst Climate Disaster	
2	**Introduction**	8
	2.1 Background and Motivation | 2.2 Problem Statement | 2.3 Objectives | 2.4 Scope	
3	**System Overview**	9
	3.1 Purpose | 3.2 Target Users | 3.3 System Architecture	
4	**Requirements Analysis**	10
	4.1 Functional Requirements | 4.2 Non-Functional Requirements	
5	**Disaster Report Submission**	13
	5.1 Disaster Report Submission | 5.2 Reporter Verification	
6	**Volunteer Management**	15
	6.1 Registration | 6.2 Medical vs Tech Volunteers | 6.3 Pre-loaded Government Volunteers	
7	**Relief Camp & Resource Management**	16
	7.1 Pre-loaded Relief Camps | 7.2 Resource Deployment Formula | 7.3 Central Warehouse Fallback	
8	**Donation System**	17
	8.1 Fund Allocation	
9	**CNIC Cooldown & Anti-Abuse System**	18
10	**Alert & Notification System**	19
	10.1 Alert Methods | 10.2 Complete Alert Trigger Table	
11	**Admin Dashboard**	20
	11.1 Authentication | 11.2 Admin Panel Options	
12	**Multi-Language Support**	21
13	**Data Persistence**	22
14	**Memory Management**	23
	14.1 unique_ptr | 14.2 Raw Pointer | 14.3 Rule of Five | 14.4 Virtual Destructors	
15	**Security Features**	29
16	**Object-Oriented Design**	

18.1 OOP Principles | 18.2 Class Overview Table

17	**Design Patterns**	36

Factory Method | Observer | Strategy | Template Method | Facade | Summary

18	**Testing & Sample Scenarios**	37
19	**Limitations**	38
20	**Future Improvements**	39
21	**Conclusion**	40
—	**References**	41

1. Problem Domain

THE REAL PROBLEM THIS SYSTEM SOLVES

Pakistan's Deadliest Disasters & The Chaos of Mismanagement

> *On the morning of August 4, 2010, Naseem Bibi stood knee-deep in floodwater outside her village in Nowshera, Khyber Pakhtunkhwa. Her daughter was burning with fever. She had been trying to reach rescue services for 16 hours. The phone lines were jammed. The local relief office had received 3,000 calls that morning — all manually recorded in a paper register. No one had a system to prioritize which call to answer first. By afternoon, the floodwaters had risen. Naseem's daughter did not survive.*
> **— Reconstructed account, 2010 Pakistan Super Floods**

1.1 Pakistan's Disaster Crisis: A Nation Without a System

Pakistan is one of the most disaster-prone countries on Earth. Between floods, earthquakes, cyclones, landslides, and droughts, barely a year passes without a catastrophe claiming hundreds of lives. But the deaths recorded in official statistics only tell half the story. The other half — the preventable half — is written in delayed responses, misrouted aid, unverified reports, and the complete absence of a unified digital coordination system.

1.1.1 The 2010 Super Floods: 20 Million Displaced, Coordination in Ruins

In July and August 2010, Pakistan witnessed the worst flooding in its recorded history. One-fifth of the country was submerged. Over 20 million people were displaced. More than 1,700 lost their lives. Yet NDMA officials admitted in post-disaster reviews that one of the greatest challenges was not the flood itself — it was knowing where to send help first.

Relief trucks arrived in some areas multiple times while adjacent villages received nothing for weeks. Donated food rotted in Lahore warehouses while Sindh families survived on contaminated floodwater. Why? Because there was no digital system that could aggregate incoming distress calls, verify their authenticity, rank them by severity, and dispatch the nearest resource automatically.

DRCS Solution: system that automatically verifies every citizen report against a national identity database, calculates a priority score based on disaster severity and reporter type, and instantly ranks all incoming reports. Resources are then dispatched to the highest-need area first – removing human bottlenecks and subjective decision-making.

1.1.2 The 2005 Kashmir Earthquake: 87,000 Dead, Help That Came Too Late

On October 8, 2005, a 7.6-magnitude earthquake struck Azad Kashmir and northern Pakistan. It remains the deadliest earthquake in Pakistan's history. Entire cities like Muzaffarabad and Balakot were leveled within seconds. The death toll climbed to approximately 87,000.

Survivors who could reach a phone had no standardized way to report their situation. Rescue teams deployed to coordinates reported by helicopter pilots, not by verified civilian distress calls. Duplicate reports of the same site wasted entire days of rescue effort. Injured survivors who needed medical volunteers first received food supply teams, while others waited for medical attention that went to already-assisted areas.

> *We had hundreds of calls. We had no way to know which were real, which were exaggerated, and which location had already been served. We were guessing.*
> **— Field Coordinator, 2005 AJK Relief Operation**

DRCS Solution: A built-in verifier scans each report for fake content and validates the reporter's CNIC and phone number before acceptance. A CNIC-based cooldown mechanism blocks duplicate submissions from the same location within 24 hours. Rescue teams receive only verified, de-duplicated, and priority-sorted reports – not raw, unverified noise.

1.1.3 Cyclone Biparjoy 2023: Warnings Issued, But Who Coordinated the Evacuation?

In June 2023, Cyclone Biparjoy made landfall near Keti Bandar, Sindh, with wind speeds exceeding 150 km/h. Pakistan's meteorological department issued warnings days in advance. PDMA Sindh activated emergency protocols. But the gap between 'warning issued' and 'civilians evacuated' was catastrophic.

Thousands of fishermen along the Makran coast did not evacuate because they never received a direct, structured alert through any digital channel. Relief camps were set up in Karachi — but volunteers had no centralized list of who had registered, what specialization they held, or which camps had sufficient supply stock. Donations poured in through various bank accounts but no system tracked which city needed what most urgently.

DRCS Solution: The alert system instantly broadcasts mass evacuation messages (simulating SMS, radio, TV, and sirens) the moment a cyclone report is submitted. All relevant agencies (NDMA, PDMA, Rescue 1122) are notified simultaneously. Volunteer availability and relief camp stock levels are tracked in real time, and donations are routed directly to the matching city camp.

1.1.4 2022 Floods: 33 Million Affected — Pakistan's Worst Climate Disaster

The monsoon floods of 2022 were described by UN Secretary-General Antonio Guterres as a "monster monsoon." One-third of Pakistan was underwater. 33 million people were affected. The Balochistan and Sindh provinces were worst hit, with entire districts cut off for weeks.

Aid workers from international organizations reported that the greatest obstacle to relief distribution was not lack of resources — it was the absence of a system that could tell them which of the hundreds of affected districts had the lowest camp stock, the highest severity, and the fewest deployed volunteers. Admin officials made decisions based on WhatsApp messages and phone calls rather than data.

DCS Solution: A single admin dashboard provides a unified interface to view all reports sorted by priority, check real-time stock of every relief camp, deploy volunteers and resources, impose lockdowns, order evacuations, and save all data persistently – enabling data-driven decisions without relying on WhatsApp or phone calls

The Recurring Pattern: Mismanagement Kills More Than the Disaster

Across every major disaster in Pakistan's history, a recurring pattern emerges:

- No unified digital reporting channel — citizens scramble to reach anyone who will listen
- No verification of incoming reports — fake, duplicate, and exaggerated calls drown real emergencies
- No priority-based resource deployment — aid is distributed by proximity or politics, not need
- No real-time volunteer management — willing helpers cannot be coordinated
- No automated agency notification — NDMA, PDMA, and Rescue 1122 operate in parallel silos
- No persistent data system — records are lost, session by session, disaster by disaster

The Disaster Relief Coordination System is designed to close every single one of these gaps. Not theoretically — with real, working C++ code built on five design patterns, four national languages, file persistence, and a complete admin management suite.

This report documents every feature of the system, explains how each module addresses a documented failure of Pakistan's disaster response infrastructure, and demonstrates the OOP principles and design patterns that make the solution extensible, maintainable, and production-ready.

2. Introduction

2.1 Background and Motivation

In disaster-prone regions, the difference between life and death often comes down to speed, coordination, and reliable information. When multiple agencies operate in isolated compartments using manual, unverified reporting channels, relief efforts become delayed, duplicate, or misdirected. Citizens have no standardized way to report emergencies, and authorities struggle to prioritize the most urgent cases. This project was motivated by the need for a unified, automated disaster coordination system – one that could replace fragmented phone calls and spreadsheets with a single platform that verifies reports, prioritizes responses, and mobilizes all relevant stakeholders simultaneously.

2.2 Objectives

- Provide citizens with a structured, verified disaster reporting interface accessible in four national languages.
- Automatically validate reporter identity, scan for fake content, and calculate a priority score for every accepted report.
- Manage relief volunteers, camps, and donations through a centralized, password-protected admin dashboard.

- Notify NDMA, PDMA, and Rescue 1122 instantly upon every alert event using the Observer pattern.
- Prevent abuse through CNIC-based cooldown rules, role conflict detection, and fake keyword filtering.
- Trigger automatic mass evacuation alerts when disaster severity demands immediate civilian action.
- Persist all data across sessions using structured file storage so no information is lost between runs.

2.3 Scope

The system is a console-based C++ application targeting disaster coordination within Pakistan. It supports seven disaster categories, multi-language menus in English, Urdu, Sindhi, and Pashto, monetary and item donations, volunteer registration, pre-loaded government data for eight major cities, and a protected admin panel with twelve operational functions.

2.4 Report Organization

Sections 2–3 establish system overview and requirements. Sections 4–5 detail OOP design principles and all five design patterns — each illustrated with a real Pakistan disaster scenario. Sections 6–14 describe every functional module with UML class excerpts and real-world context. Sections 15–20 cover memory management, security, testing, limitations, future improvements, and the conclusion.

3 SYSTEM OVERVIEW

3.System Overview

3.1 Purpose

The Disaster Relief Coordination System (DRCS) is a C++ console application that acts as a bridge between affected citizens who report emergencies and the government agencies responsible for deploying relief. It automates verification, prioritization, alerting, and resource tracking — tasks that are otherwise performed manually, inconsistently, and slowly.

3.2 Target Users

User Role	Description	System Access
Civilian	Any Pakistani citizen with a valid 13-digit CNIC and registered phone number	Submit reports, register as volunteer, make donations, view emergency numbers
Officer	Government or field officer verified via a 3-digit officer code	All civilian access + elevated priority score on submitted reports
Admin	System administrator authenticated via a hashed password	Full access: view all reports, deploy resources, manage camps, impose lockdown, order/lift evacuations, mark reports resolved

3.3 System Architecture

The system is organized into three logical layers:

- Presentation Layer: Console menus, multi-language support via SystemConfig, and formatted output functions.
- Business Logic Layer: DisasterReport, Verifier, PriorityStrategy, AlertSystem, and AdminDashboard — all design-pattern driven.
- Data Layer: Six pipe-delimited text files storing reports, volunteers, camp stock, fund balance, CNIC history, and evacuation records.

4 REQUIREMENTS ANALYSIS

4. Requirements Analysis

4.1 Functional Requirements

ID	Requirement	Module
FR-01	Citizens submit disaster reports with location, identity, disaster type, severity, and description	Report Submission

ID	Requirement	Module
FR-02	Reporter CNIC and phone number validated before a report is accepted	Verifier / Reporter
FR-03	Officer reporters must provide a valid 3-digit officer code	Reporter
FR-04	Reports containing any of 15 fake keywords are automatically rejected	Verifier
FR-05	Verified reports receive a calculated priority score	PriorityStrategy
FR-06	Disasters meeting evacuation thresholds trigger automatic mass citizen alerts	AlertSystem
FR-07	Citizens register as Medical or Tech volunteers with auto-generated IDs	Volunteer
FR-08	Monetary and item donations accepted and routed to matching city camps	DonationManager / ItemDonation
FR-09	Admin can view reports, deploy resources, lockdown, order/lift evacuations	AdminDashboard
FR-10	NDMA, PDMA, and Rescue 1122 notified on every alert event	Observer Pattern
FR-11	CNIC-based cooldown prevents duplicate or abusive reporting	CNIC Cooldown
FR-12	All data persists across sessions via file storage	File Persistence

4.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Admin password stored as djb2 hash; officer code gate for privileged reporting
NFR-02	Usability	Multi-language menus in English, Urdu, Sindhi, and Pashto selected at startup
NFR-03	Reliability	CNIC type-conflict detection via cnicTypeMap prevents role spoofing
NFR-04	Performance	Report submission and verification complete within the same interactive session
NFR-05	Maintainability	Five design patterns ensure each module is independently extensible
NFR-06	Portability	Compiles and runs on both Windows (cls) and Unix/Linux (clear) platforms

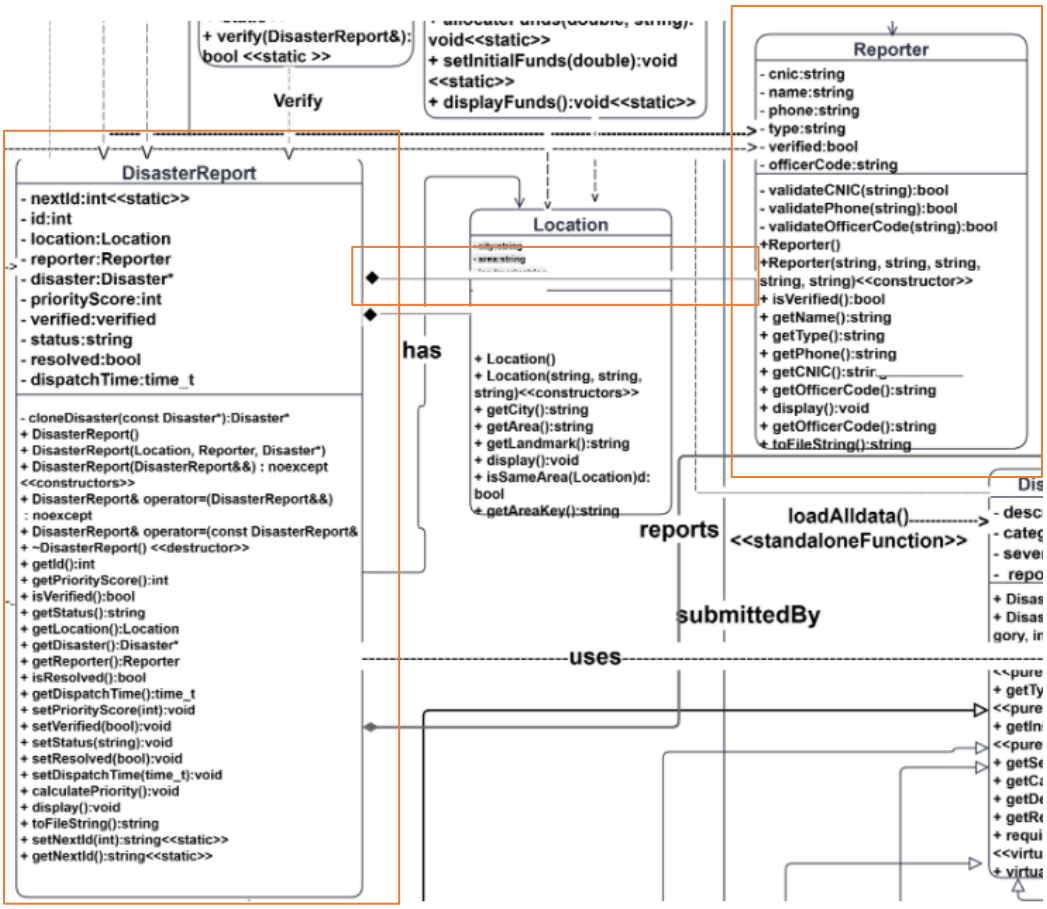
5 DISASTER REPORT SUBMISSION

5. System Modules

5.1 Disaster Report Submission

Real-World Context: After the 2005 Kashmir earthquake, survivors who could reach any telephone had no standardized way to describe their situation. Different operators asked different questions. There was no uniform format for location, severity, or disaster type — so no two reports were comparable. DRCS solves this with a structured, mandatory submission form.

Report submission is the primary citizen-facing feature. The user provides location (city, area, landmark), personal details (name, CNIC, phone, reporter type), and disaster details (type, severity 1–10, description). The system constructs a Location, Reporter, and Disaster* object, then passes them to DisasterReport. Verifier::verify() is called immediately after construction. If the report passes all checks, it is added to allReports and the CNIC submission is logged. If the disaster type requires evacuation at the reported severity, a mass evacuation broadcast is triggered automatically.

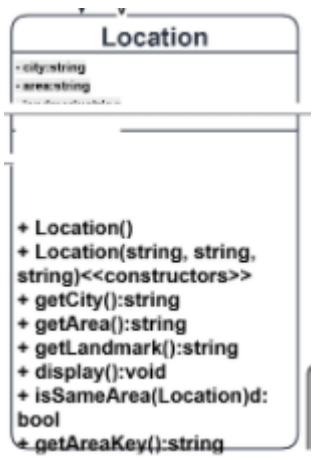



```
========== SUBMIT DISASTER REPORT ==========
City: Karachi
Area/Sector: Saddar
Nearby Landmark: Gul Plaza

--- Your Information ---
Full Name: Zainab Kinza
CNIC (13 digits without dashes): 4220187336450
Phone (03xxxxxxxxx): 03006255987
Type (Civilian/Officer): civilian

--- Disaster Details ---
Disaster Types: FLOOD, EARTHQUAKE, CYCLONE, LANDSLIDE, FIRE, DROUGHT, PANDEMIC
Disaster Type: fire
Severity (1-10): 9
Description: The gul plaza building is on fire.
```

5.2 Reporter Verification

Real-World Context: During the 2010 floods, relief hotlines were overwhelmed by prank calls, repeat calls, and exaggerated reports. Aid workers spent hours manually cross-referencing calls. DRCS's Verifier class automates this entire process, rejecting invalid reports before they ever enter the system.

Field	Validation Rule	Failure Response
CNIC	Exactly 13 digits, no leading zero, not all identical digits	Report rejected: Invalid CNIC
Phone Number	11 digits starting with 03, or 10 digits starting with 3	Report rejected: Invalid Phone Number
Officer Code	Exactly 3 digits equal to OFFICER_SECRET_CODE constant (131)	Report rejected: Invalid Code
CNIC Type Conflict	A CNIC registered as Civilian cannot re-register as Officer in the same session	Report rejected: CNIC already registered



```
========== SUBMIT DISASTER REPORT ==========
City: Lahore
Area/Sector: DHA phase 5
Nearby Landmark: Central park

--- Your Information ---
Full Name: Saita Ahuja
CNIC (13 digits without dashes): 4210166678
Phone (03xxxxxxxxx): 030054667893
Type (Civilian/Officer): civilian

[ERROR] Invalid CNIC!

Verification failed!

Press Enter to continue...
```

```
========== SUBMIT DISASTER REPORT ==========
City: Karachi
Area/Sector: University road
Nearby Landmark: Karachi University

--- Your Information ---
Full Name: Bilal Ahmed
CNIC (13 digits without dashes): 4210112345678
Phone (03xxxxxxxxx): 03007689556
Type (Civilian/Officer): officer
Officer Code (3 digits): 777

[ERROR] This CNIC is already registered in the system.

Verification failed!

Press Enter to continue...
```

6. Volunteer Management

Real-World Context: After Cyclone Biparjoy (2023), thousands of volunteers registered through informal WhatsApp groups. Coordinators had no database of specializations, no real-time availability tracking, and no standardized deployment protocol. Willing helpers sat idle while camps 50 km away desperately needed medical staff. DRCS's Volunteer module solves all three problems.

6.1 Registration Process

Citizens register as volunteers by providing their name, CNIC (13 digits, validated), phone number, type (Medical or Tech), and specialization. The system auto-generates a unique ID in the format GOVxxx starting from GOV016 onwards, following the 15 pre-loaded government volunteers. Volunteers are stored as unique_ptr<Volunteer> in the allVolunteers vector, ensuring automatic memory cleanup.

6.2 Medical vs Tech Volunteers

- Medical Volunteers: Doctors, nurses, paramedics. preDeployCheck() prepares a medical kit; performDeploy() announces medical deployment.
- Tech Volunteers: Drivers, rescue specialists, engineers, logistics staff. preDeployCheck() prepares field equipment; performDeploy() announces technical deployment.

6.3 Pre-loaded Government Volunteers

ID	Name	Type	Specialization
GOV001	Dr. Ahmed Khan	Medical	Doctor
GOV002	Dr. Fatima Ali	Medical	Doctor
GOV003	Nurse Sara	Medical	Nurse
GOV004	Dr. Usman Chaudhry	Medical	Doctor
GOV005	Nurse Ayesha	Medical	Nurse
GOV006	Dr. Bilal Ahmed	Medical	Surgeon
GOV007	Commander Raza	Tech	Rescue Specialist
GOV008	Engineer Kashif	Tech	Heavy Machinery
GOV009	Squad Leader Tariq	Tech	Rescue
GOV010	Driver Zafar	Tech	Driver
GOV011	Technician Ali	Tech	Technician
GOV012	Logistics Incharge	Tech	Logistics
GOV013	Dr. Sana Mir	Medical	Pediatrician
GOV014	Nurse Kiran	Medical	Nurse

ID	Name	Type	Specialization
GOV015	Field Commander	Tech	Commander

7 RELIEF CAMP & RESOURCE MANAGEMENT

7. Relief Camp and Resource Management

Real-World Context: In the 2010 floods, the UN reported that aid was wasted because no one tracked which camps had surplus and which were empty. Entire truckloads of food rotted in Lahore warehouses while Sindh families had nothing. DRCS maintains real-time stock counts for every camp and automatically routes resources to the highest-priority report's city.

7.1 Pre-loaded Relief Camps

Camp Name	City	Initial Food (units)	Initial Medicine (units)
Central Camp Karachi	Karachi	5,000	2,000
Lahore Relief Hub	Lahore	5,000	2,000
Islamabad Emergency Center	Islamabad	5,000	2,000
Peshawar Base Camp	Peshawar	5,000	2,000
Quetta Relief Camp	Quetta	5,000	2,000
Multan Central Camp	Multan	5,000	2,000
Gilgit Base Camp	Gilgit	5,000	2,000
Central Warehouse	Central	5,000	2,000

7.2 Resource Deployment Formula

Food Required = Severity × 100 units | Medicine Required = Severity × 50 units | Funds Allocated = Rs. 50,000

7.3 Central Warehouse Fallback

If an item donation targets a city with no matching camp, it is automatically routed to the Central Warehouse. The same fallback applies during resource deployment — if no camp exists in the disaster's city, the Central Warehouse supplies the resources. This mirrors the real lesson from 2022 when Balochistan's remote districts had no local depot — the system automatically escalates to the national warehouse.

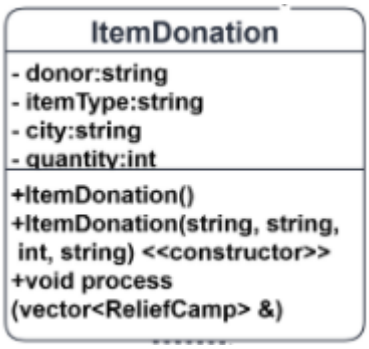
8. Donation System

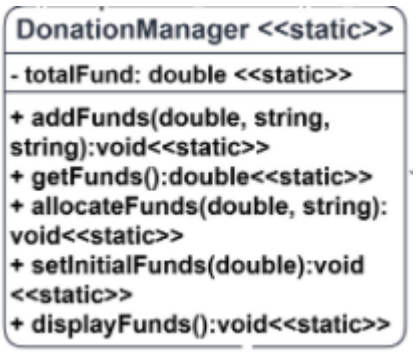
Real-World Context: During the 2022 floods, Pakistan received over $160 million in international aid. But donors had no visibility into which city needed food vs. medicine, and aid organizations had no automated routing. Donations piled up in central accounts while specific districts ran out of medicine. DRCS's donation system routes every contribution directly to the matching city camp.

Donation Type	Required Input	Processing Logic	Output
Monetary	Donor name, CNIC, amount (Rs.)	DonationManager::addFunds() increments totalFund	Updated fund balance displayed
Item — Food	Donor name, quantity, target city	ItemDonation::process() finds city camp and calls addFood(qty)	Delivery confirmation or Central Warehouse redirect
Item — Medicine	Donor name, quantity, target city	ItemDonation::process() finds city camp and calls addMedicine(qty)	Delivery confirmation or Central Warehouse redirect

8.1 Fund Allocation

The admin allocates funds from the relief pool via DonationManager::allocateFunds(amount, purpose). Automatic allocations occur during lockdowns (Rs. 100,000), evacuation orders (Rs. 200,000), and resource deployments (Rs. 50,000). The system is seeded with Rs. 5,000,000 of government funds on first run.

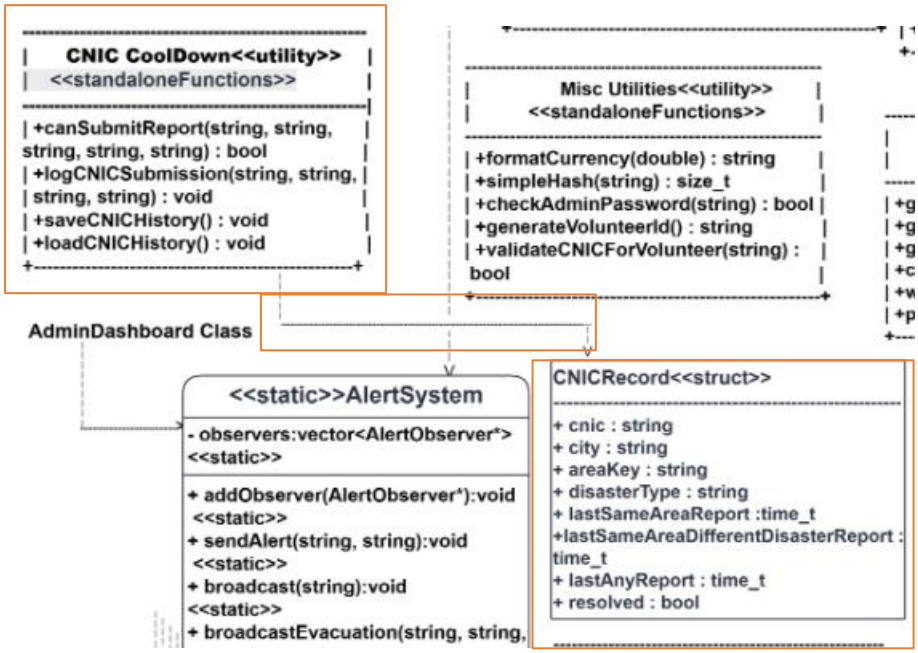




9. CNIC Cooldown and Anti-Abuse System

Real-World Context: During every major Pakistani disaster, relief hotlines are paralyzed by prank calls, politically motivated false reports, and individuals calling repeatedly to jump the queue. In the 2010 floods, NDMA estimated that up to 30% of incoming calls were either duplicate or fabricated. DRCS uses CNIC-based tracking to block all three abuse patterns automatically.

Scenario	Condition	Block Duration	Message Shown to User
Same CNIC, same area, same disaster	Re-submitting within 24 hours	24-hour block	Help is already on the way. Next update allowed after X hours Y minutes.
Same CNIC, same area, different disaster	Any re-attempt	Permanent for this combination	Suspicious activity detected. Cannot report different disaster from same location.
Same CNIC, different city	Re-submitting within 30 days	30-day block	Report already submitted. Next report allowed after X days.



```
========== SUBMIT DISASTER REPORT ==========
City: Karachi
Area/Sector: University road
Nearby Landmark: Karachi University

--- Your Information ---
Full Name: Fatima Khan
CNIC (13 digits without dashes): 4210112345678
Phone (03xxxxxxxxx): 03004567890
Type (Civilian/Officer): civilian

--- Disaster Details ---
Disaster Types: FLOOD, EARTHQUAKE, CYCLONE, LANDSLIDE, FIRE, DROUGHT, PANDEMIC
Disaster Type: earthquake

[BLOCKED] Report already submitted with this CNIC.
   Next report allowed after 29 days.
   Call 1122 or NDMA: 1131 for emergencies.

Press Enter to continue...
```

10 ALERT & NOTIFICATION SYSTEM

10. Alert and Notification System

Real-World Context: In every documented Pakistan disaster, the time between a verified threat and coordinated agency response was measured in hours, not minutes. The 2023 Cyclone Biparjoy evacuation was delayed because NDMA, PDMA, and Rescue 1122 each had to be contacted through separate channels. DRCS's AlertSystem notifies all three simultaneously, automatically, within milliseconds of a verified report.

10.1 Alert Methods

- sendAlert(area, message): Targeted alert for a specific area. Used for lockdowns and evacuation orders.
- broadcast(message): System-wide message sent to all observers with area = ALL. Used after deployments and escalations.
- broadcastEvacuation(city, area, type, severity): Full mass-alert block simulating SMS, radio, TV, and siren activation for all citizens in the affected area.

10.2 Complete Alert Trigger Table

Trigger	Method Called	Message	Recipients
Verified report with evacuation flag	broadcastEvacuation()	EMERGENCY EVACUATION ALERT to all citizens	All citizens (simulated) + NDMA, PDMA, Rescue 1122
Admin: Impose Lockdown	sendAlert() + broadcast()	LOCKDOWN imposed. Stay indoors.	NDMA, PDMA, Rescue 1122
Admin: Order Evacuation	broadcastEvacuation() + sendAlert()	EVACUATION ORDER issued by Admin.	All citizens + all agencies

Trigger	Method Called	Message	Recipients
Admin: Lift Evacuation	sendAlert() + broadcast()	EVACUATION ORDER LIFTED. Area is now safe.	NDMA, PDMA, Rescue 1122
Admin: Deploy Resources	broadcast()	Resources deployed to [city]	NDMA, PDMA, Rescue 1122
Admin: Follow-Up Escalation	broadcast()	URGENT: Report #X — Resources not reached!	NDMA, PDMA, Rescue 1122

11 ADMIN DASHBOARD

11. Admin Dashboard

Real-World Context: The 2022 flood response required district coordinators to simultaneously manage dozens of subsystems through phone calls and spreadsheets. DRCS's AdminDashboard (Facade Pattern) consolidates every operation into a single, password-protected interface — no phone calls, no spreadsheets, no chaos.

11.1 Authentication

The admin password is validated using the djb2 hashing algorithm (simpleHash()). The stored hash corresponds to admin123. At login, the entered password is hashed at runtime and compared — the plaintext is never stored or compared directly.

11.2 Admin Panel Options

#	Function	What It Does
1	Dashboard Stats	Total reports, verified/resolved counts, fund balance, volunteer availability, camp stock, active evacuations
2	View All Reports	Full details of every submitted report — status, priority score, location, disaster type, instructions
3	View Resources	All volunteers with availability, all camps with current stock levels, total relief fund
4	Impose Lockdown	Sends lockdown alert to specified area; broadcasts to agencies; allocates Rs. 100,000
5	Order Evacuation	Issues mass evacuation for specified city/area/disaster; logs record; allocates Rs. 200,000
6	Lift Evacuation	Shows active evacuations; marks selected as lifted; notifies all agencies area is safe
7	View Evacuations Log	All past and present evacuation orders with timestamp and current status

#	Function	What It Does
8	Deploy Resources	Sends food, medicine, funds, and a volunteer to the highest-priority unresolved report
9	Follow-Up Resources	Triggered 2 hours after dispatch (simulated as 1 minute in project); confirms arrival — re-deploys and escalates via broadcast if not confirmed
10	Add Relief Camp	Creates a new ReliefCamp with zero stock for a specified name and city
11	Mark Report Resolved	Sets report status to RESOLVED by ID; removes from future deployment queues
12	Save & Exit Admin	Persists all current data to files and exits the admin panel

11.3 Automatic Evacuation on Report Submission

When a verified report is submitted and the disaster type meets its severity threshold — Cyclone (always), Flood/Earthquake (severity ≥ 6/5), Fire/Landslide (severity ≥ 7), Pandemic (severity ≥ 8) — the system automatically issues a mass evacuation broadcast to SMS, radio, TV, and sirens without any admin intervention. The evacuation record is logged and persisted immediately

```
+================================+
|        ADMIN PANEL             |
+================================+
| 1.  Dashboard Stats            |
| 2.  View All Reports           |
| 3.  View Resources             |
| 4.  Impose Lockdown            |
| 5.  Order Evacuation           |
| 6.  Lift Evacuation            |
| 7.  View Evacuations Log       |
| 8.  Deploy Resources           |
| 9.  Follow Up Resources        |
| 10. Add Relief Camp            |
| 11. Mark Report Resolved       |
| 12. Save & Exit Admin          |
+================================+
Choice: |
```

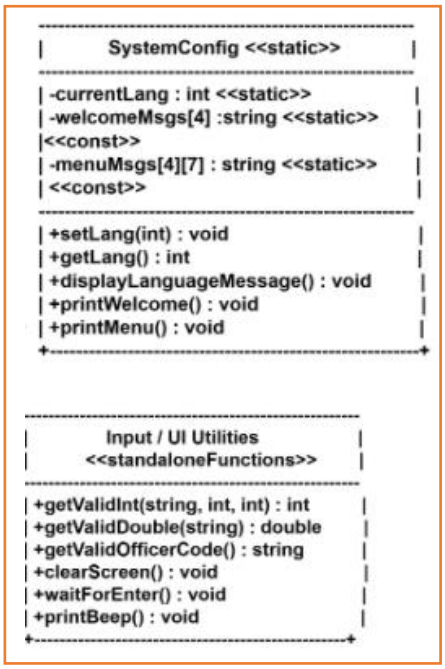
Admin panel display

12 MULTI-LANGUAGE SUPPORT

12. Multi-Language Support

Real-World Context: Pakistan is a multilingual nation. In the 2005 earthquake, Pashto-speaking survivors in Azad Kashmir struggled to communicate with Urdu-speaking operators. In 2010 floods, Sindhi-speaking villages in Sindh had no way to navigate Urdu-only relief hotlines. DRCS is the first fictional system to serve disaster victims in their own language, removing the communication barrier entirely.

Code	Language	Welcome Message	Menu Sample
1	English	Welcome to Disaster Relief System - Pakistan	1. Submit Report
2	Urdu	Khushamdeed - Pakistan Disaster Relief System	1. Report jama karein
3	Sindhi	Bhalli Kare Aaya - Disaster Relief System	1. Report jama kayo
4	Pashto	Har kala rasha - Disaster Relief System	1. Report darj ka



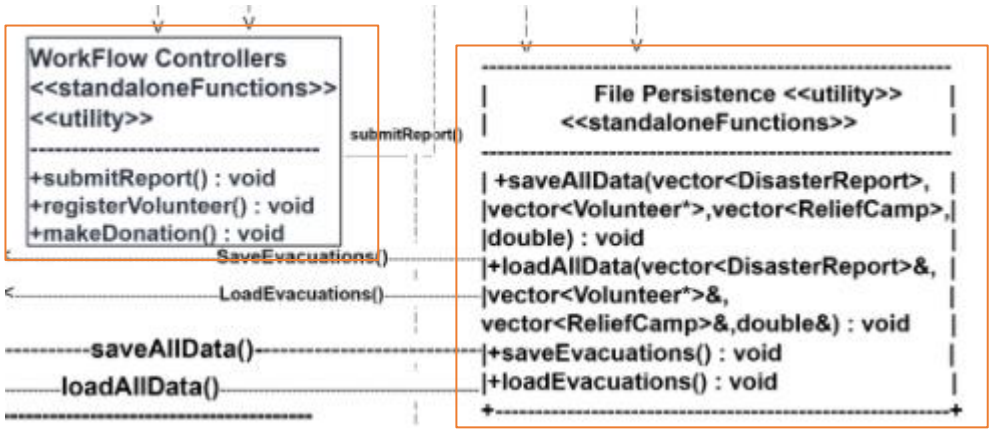
13 DATA PERSISTENCE

13. Data Persistence

Real-World Context: After the 2005 earthquake, NDMA's paper-based records were destroyed when their own offices were damaged. Survivor lists, camp rosters, and donation records were lost. DRCS persists every piece of data to six structured files at the end of every operation. If the system restarts, all data is restored perfectly.

File	Contents	Sample Format
reports_data.txt	All disaster reports	id|city|area|landmark|cnic|name|phone|type|code|verified|disasterType|severity|desc|verStr|status|priority|resolved
volunteers_data.txt	All registered volunteers	name|id|cnic|phone|type|available|specialization

File	Contents	Sample Format
camps_data.txt	Relief camp stock levels	campName|city|foodStock|medicineStock
funds_data.txt	Current total relief fund	5000000
cnic_history.txt	CNIC cooldown records	cnic|city|areaKey|disasterType|sameAreaTime|diffDisasterTime|anyTime|resolved
evacuations_data.txt	Evacuation order log	area|city|disasterType|severity|issuedAt|lifted



14 MEMORY MANAGEMENT

14. Memory Management

14.1 unique_ptr for Volunteers

All volunteers in allVolunteers are stored as unique_ptr<Volunteer>. This guarantees automatic cleanup when the vector is destroyed, and prevents double-deletion. Ownership is transferred with std::move() during insertion.

14.2 Raw Pointer for Disaster*

DisasterReport owns its Disaster* as a raw pointer. Manual deletion is performed in the destructor. The private helper cloneDisaster() creates deep copies for the copy constructor and copy assignment operator. Move operations transfer ownership and null out the source pointer to avoid double-deletion.

14.3 Rule of Five in DisasterReport

DisasterReport implements all five special member functions: destructor (deletes Disaster*), copy constructor (deep clones via cloneDisaster()), copy assignment operator (deletes old, deep clones new), move constructor (transfers pointer, nulls source), move assignment operator (same with self-assignment check).

14.4 Virtual Destructors

Both Disaster and Volunteer declare virtual destructors. This ensures that when a derived object (e.g., FloodDisaster or MedicalVolunteer) is deleted through a base class pointer, the correct derived destructor runs — preventing resource leaks throughout the polymorphic hierarchies.

15 SECURITY FEATURES

15. Security Features

Real-World Context: False disaster reports have been documented across multiple Pakistani disasters — politically motivated exaggerations, rivals falsely reporting competing areas as disaster zones to redirect aid, and pranksters flooding hotlines. DRCS implements six distinct security mechanisms to combat every known abuse pattern.

Threat	Mechanism	Where in Code
Fake or prank reports	15 fake keyword scan in Verifier::verify()	Verifier class, fakeWords[15] array
Invalid reporter identity	CNIC (13 digits, no leading zero, not all same) and phone format checks	Reporter constructor — validateCNIC(), validatePhone()
Unauthorized officer reporting	3-digit officer code compared to OFFICER_SECRET_CODE constant	Reporter::validateOfficerCode()
CNIC role conflict	cnicTypeMap prevents same CNIC registering as both Civilian and Officer	Reporter constructor, global cnicTypeMap
Duplicate / abusive reporting	CNIC cooldown: 24h same area, 30d different city, permanent suspicious block	canSubmitReport(), cnicHistory vector
Unauthorized admin access	djb2 hash comparison — plaintext never stored	simpleHash(), checkAdminPassword()

```
========== SUBMIT DISASTER REPORT ==========
City: Bahawalpur
Area/Sector: Model Town
Nearby Landmark: Rose Garden

--- Your Information ---
Full Name: Masood Baig
CNIC (13 digits without dashes): 4210167895642
Phone (03xxxxxxxxx): 03017639886
Type (Civilian/Officer): officer
Officer Code (3 digits): 634

[ERROR] Invalid Code!

Verification failed!

Press Enter to continue...
```

```
Admin Password: abc111
Wrong password!
```

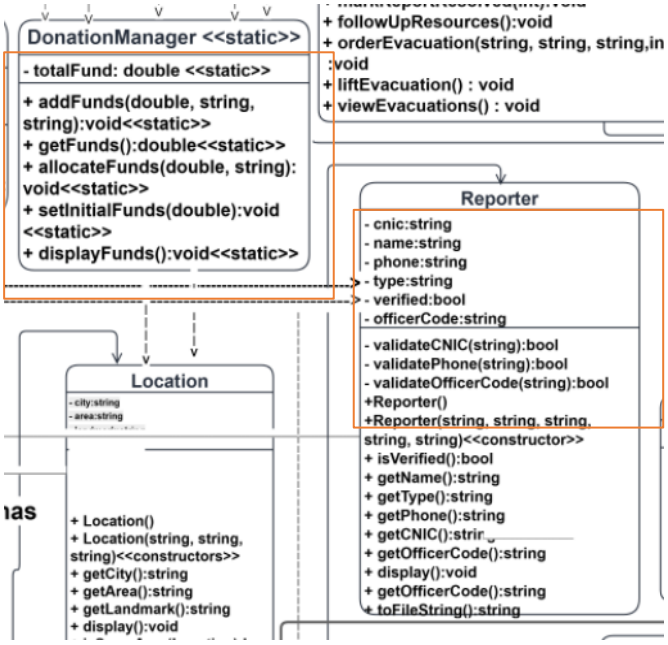
16. Object-Oriented Design

16.1 OOP Principles Applied

16.1.1 Encapsulation

Every class in DRCS keeps its internal data private and exposes only what is necessary through public methods. The Reporter class stores cnic, phone, name, and verified as private members. Validation happens internally through validateCNIC() and validatePhone() — external code can never inject an invalid identifier directly. DonationManager similarly protects its totalFund as a private static member, accessible only through addFunds(), allocateFunds(), and getFunds(). DisasterReport's priority score is calculated internally by calculatePriority() and never set from outside.

> Real-World Parallel: In the 2010 floods, unverified data entered Pakistan's relief tracking systems directly through informal channels — the equivalent of public member variables. DRCS enforces private validation so only verified, correctly formatted data enters the system.



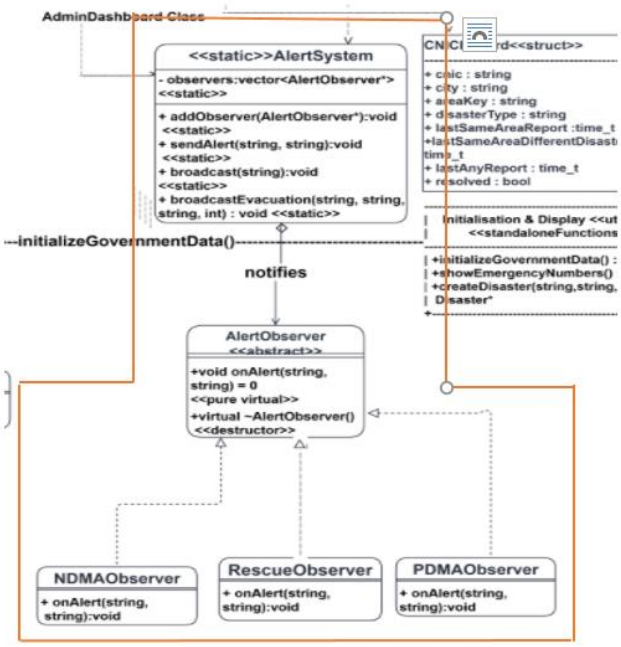
16.1.2 Inheritance

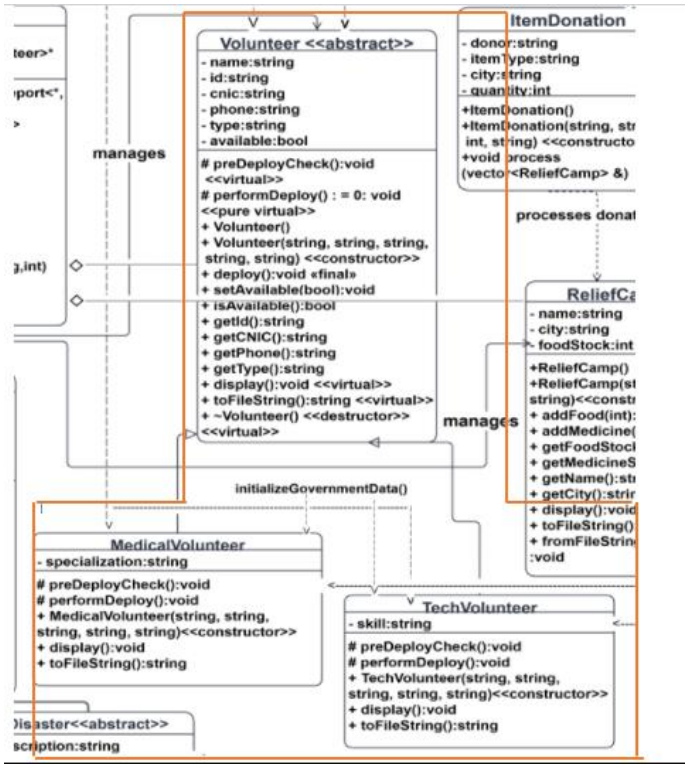
DRCS uses some major inheritance hierarchies. The Disaster abstract base class defines four pure virtual methods — getWeight(), getTypeName(), getInstructions(), and requiresEvacuation() — which seven concrete subclasses (FloodDisaster, EarthquakeDisaster, CycloneDisaster, LandslideDisaster, FireDisaster, DroughtDisaster, PandemicDisaster) inherit and override with type-specific behavior. Each disaster type returns its own urgency weight — EarthquakeDisaster returns 50, PandemicDisaster returns 55 — and defines its own evacuation threshold.

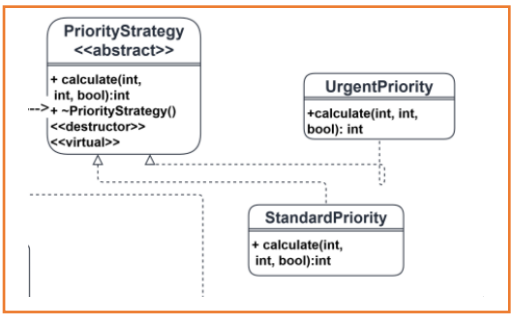
The second hierarchy is the Volunteer abstract base, from which MedicalVolunteer and TechVolunteer inherit. Each overrides preDeployCheck() and performDeploy() with specialized behavior — medical volunteers prepare medical kits, tech volunteers prepare field equipment.

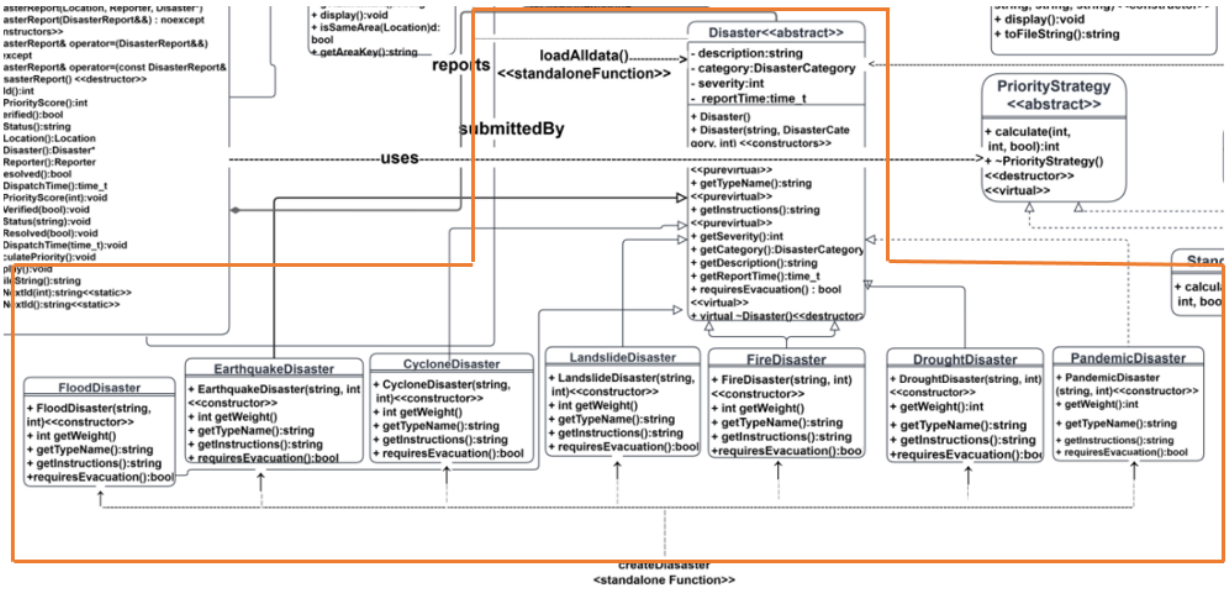
DRCS also uses inheritance in its priority calculation system through the PriorityStrategy abstract class. StandardPriority and UrgentPriority inherit from PriorityStrategy and override the calculate() method with different urgency formulas, allowing the system to dynamically switch priority logic at runtime.

The emergency alert system also uses inheritance, where AlertObserver acts as an abstract parent class and NDMAObserver, RescueObserver, and PDMAObserver inherit from it and override onAlert() for customized notifications.





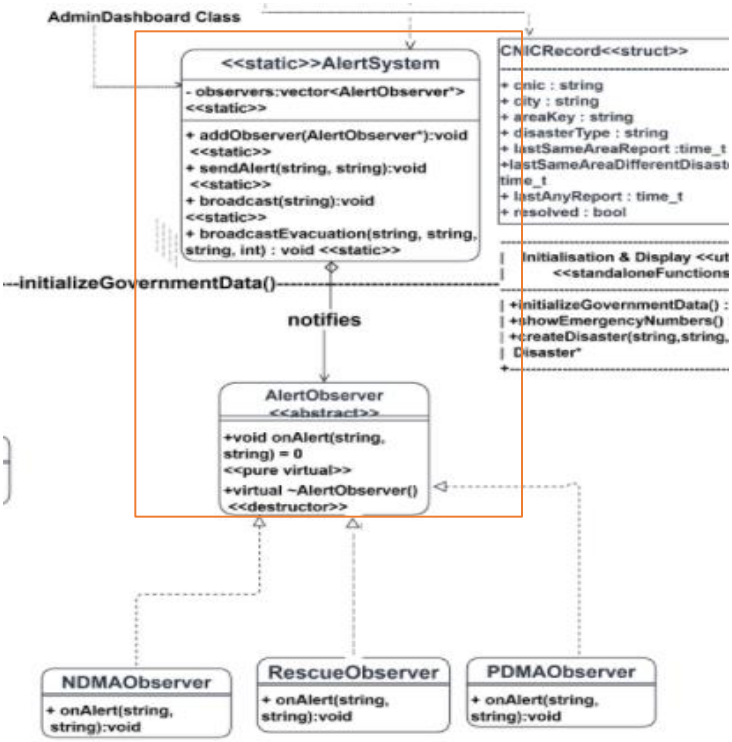




16.1.3 Polymorphism

Polymorphism is achieved throughout DRCS via base class pointers. DisasterReport holds a Disaster* pointer and calls disaster->getTypeName() and disaster->requiresEvacuation() without knowing the concrete type at compile time — the correct version executes at runtime based on the actual object. The createDisaster() factory function returns a Disaster* that transparently points to any of the seven subclasses.

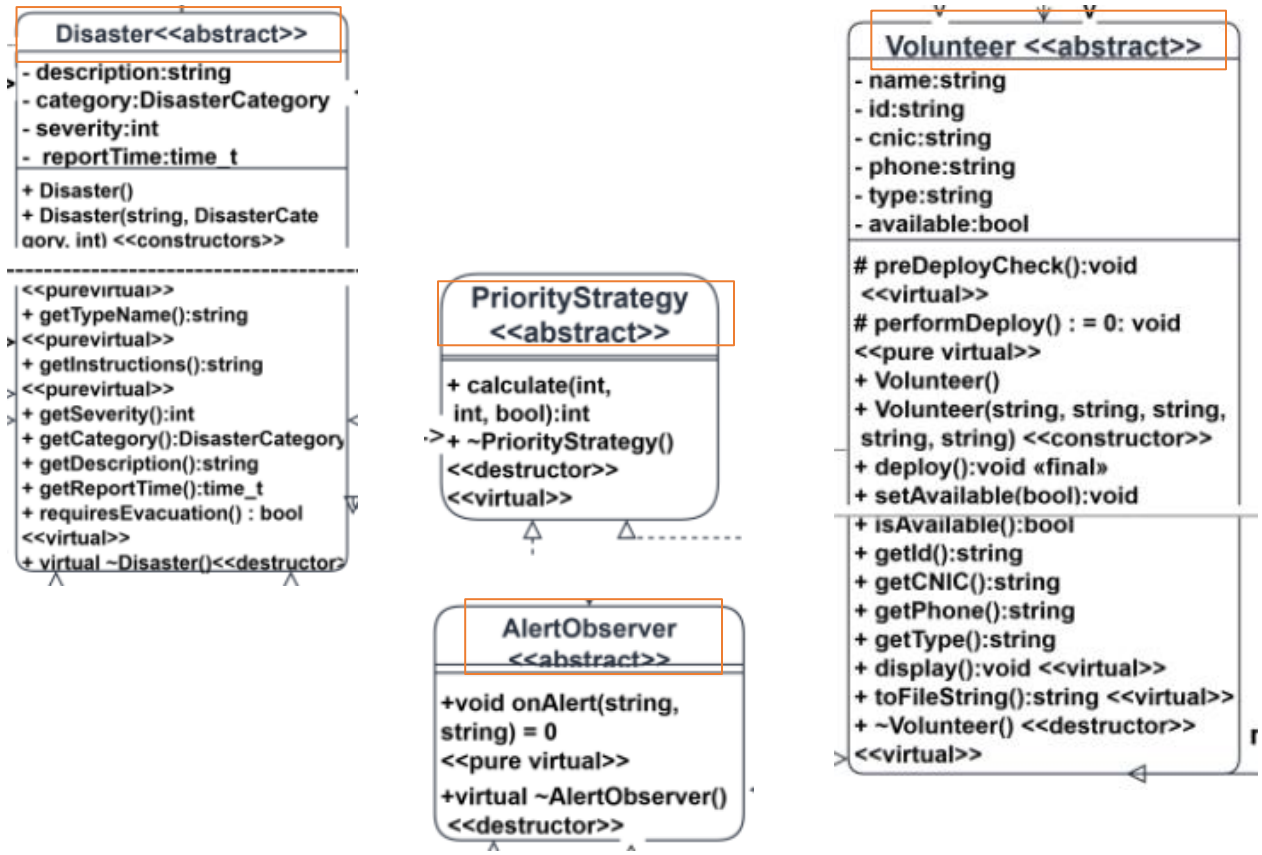
The Observer pattern in AlertSystem is another example — it holds a vector<AlertObserver*> and dispatches onAlert() to all registered observers (NDMAObserver, RescueObserver, PDMAObserver) through the same base pointer, without knowing concrete types. The Volunteer hierarchy uses the Template Method pattern where deploy() is marked final in the base class but calls performDeploy() which resolves polymorphically at runtime.



16.1.4 Abstraction

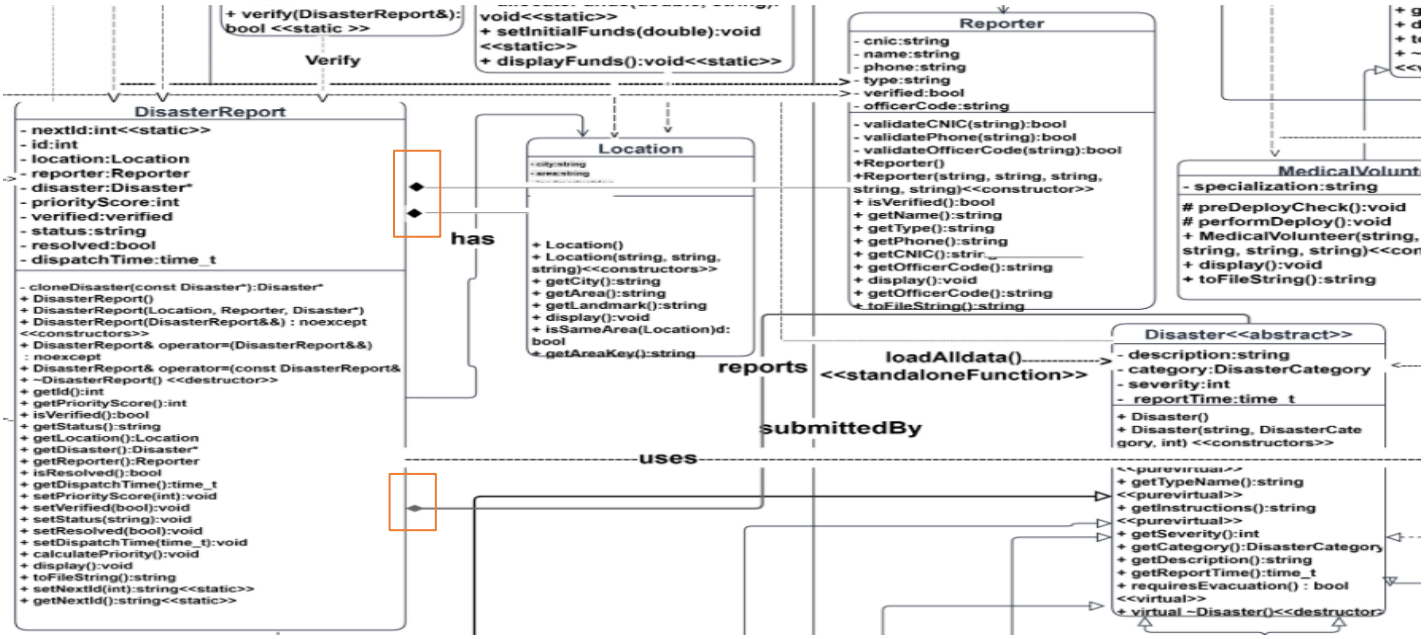
Abstraction in DRCS means hiding complex multi-step operations behind clean, single-purpose interfaces. AdminDashboard::deployResources() internally coordinates four subsystems — resource allocation from relief camps, fund deduction via DonationManager, volunteer deployment, and alert broadcasting — but the caller in main() simply invokes one method. Verifier::verify() hides keyword scanning, reporter validation, and priority calculation behind a single static call. The AlertSystem abstracts all agency notification behind sendAlert() and broadcast() — callers never deal with individual observers directly.

Abstract classes themselves enforce abstraction: Disaster, Volunteer, PriorityStrategy, and AlertObserver are all abstract — they define *what* must exist without specifying *how*, forcing concrete subclasses to provide implementations.



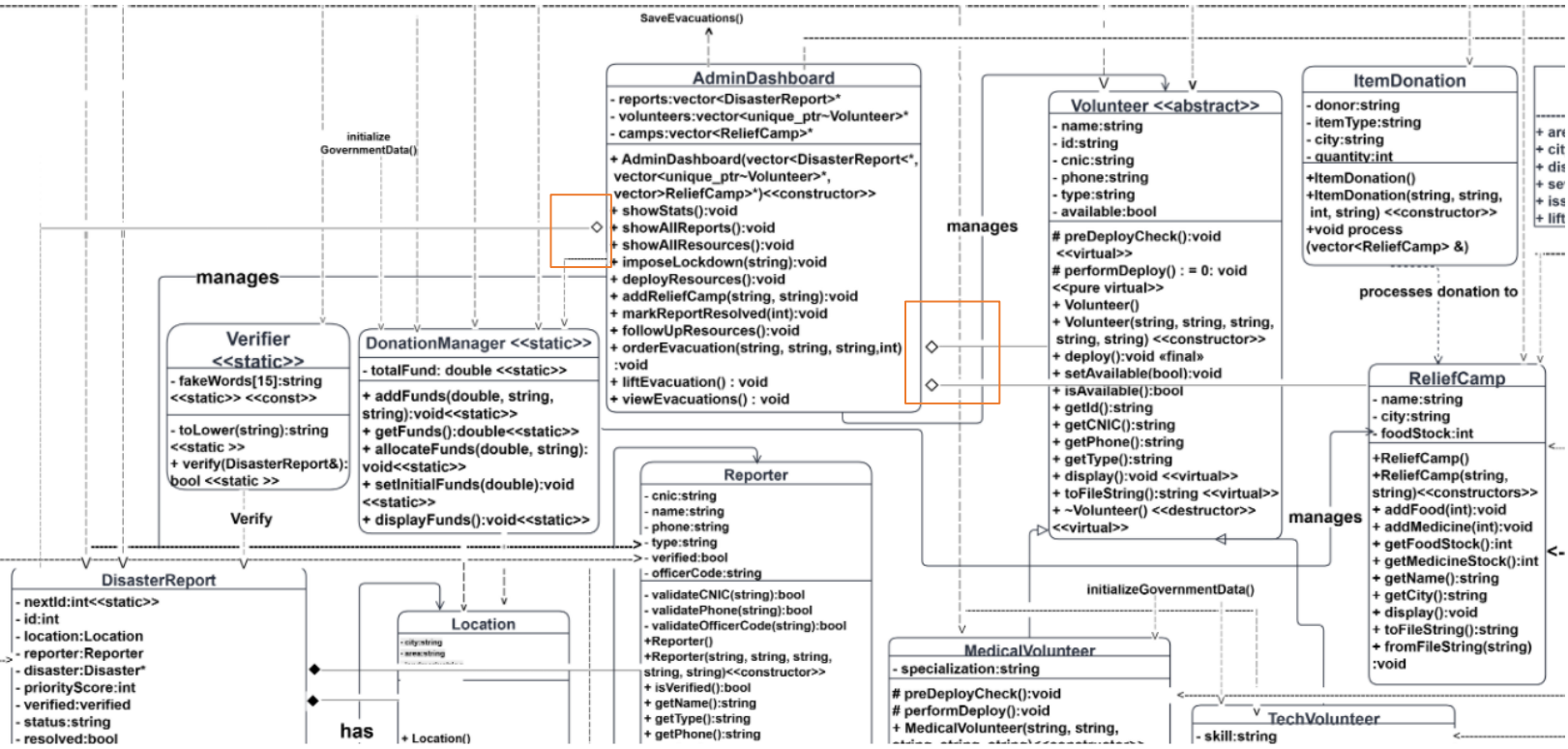
16.1.5 Composition

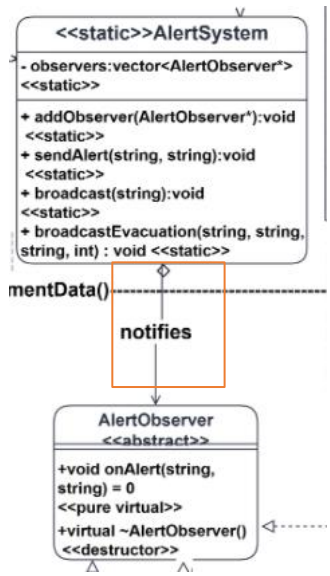
Composition represents a "owns and is responsible for" relationship — if the owner is destroyed, the owned object is destroyed with it. While implemented as a pointer (necessary because Disaster is an abstract base class requiring polymorphism), this still represents composition because the DisasterReport creates, owns, and destroys the Disaster object. In DRCS, DisasterReport is the clearest example. It owns a Disaster* pointer and is solely responsible for its memory — the destructor explicitly calls delete disaster. It also contains Location and Reporter as value members (not pointers), meaning they live and die with the DisasterReport object itself. The copy constructor performs a deep copy via cloneDisaster() to ensure each report owns its own independent Disaster object — this is the Rule of Five in action, implemented precisely because of composition ownership.



16.1.6 Aggregation

Aggregation is a weaker "uses but does not own" relationship — the container holds references or pointers to objects that exist independently and are managed elsewhere. AdminDashboard is the prime example in DRCS. Its constructor takes pointers to the global allReports, allVolunteers, and allCamps vectors — it never creates or deletes these objects. If AdminDashboard were destroyed, all the reports, volunteers, and camps would continue to exist in their global vectors untouched. Similarly, AlertSystem holds a vector<AlertObserver*> of observers like NDMAObserver, RescueObserver, and PDMAObserver — these observer objects are created externally in main() and merely registered with AlertSystem, which never deletes them.





16.1.7 Dependency (Between Classes)

Dependency is the loosest relationship — a class temporarily uses another class, typically as a method parameter or local variable, without holding any long-term reference to it. Several key dependencies exist between classes in DRCS.

Verifier depends on DisasterReport — its verify() method takes a DisasterReport& as a parameter, uses it, and has no member variable of that type. ItemDonation depends on ReliefCamp — its process() method receives a vector<ReliefCamp>& and routes donations into camps, but ItemDonation holds no camp reference after the call ends. DisasterReport depends on StandardPriority — inside calculatePriority(), a local StandardPriority object is created, used for one calculation, and destroyed — purely temporary. AdminDashboard depends on AlertSystem, DonationManager, and EvacuationRecord — it calls their static methods or creates temporary records inside its own methods without storing them as members.

16.1.8 Dependency (Standalone Functions)

Several standalone functions in DRCS act as workflow controllers and utility layers, and they depend on multiple classes to carry out their work. submitReport() is the most complex — it temporarily constructs Location, Reporter, and Disaster objects, passes them into a new DisasterReport, calls Verifier::verify() on it, and if approved, triggers AlertSystem::broadcastEvacuation() and logs an EvacuationRecord. None of these objects are stored inside the function permanently — all relationships are transient dependencies.

registerVolunteer() depends on MedicalVolunteer and TechVolunteer, constructing the appropriate one and pushing it into the global vector. makeDonation() depends on both DonationManager for monetary donations and ItemDonation for physical ones, which in turn depends on ReliefCamp.

The file persistence functions saveAllData() and loadAllData() depend on nearly every class in the system — DisasterReport, Volunteer, ReliefCamp, DonationManager, Location, Reporter, MedicalVolunteer, and TechVolunteer — reading or writing their state to disk. initializeGovernmentData() depends on ReliefCamp, MedicalVolunteer, TechVolunteer, and DonationManager to populate the system's initial state on first run.

16.2 Class Overview Table

Class / Component	Type	Responsibility
Disaster	Abstract Base	Defines interface for weight, type name, instructions, and evacuation flag
Location	Concrete	Stores city, area, landmark; provides area comparison for CNIC cooldown
Reporter	Concrete	Validates and stores CNIC, phone, name, type, officer code during construction; private validation methods
Disaster	Abstract Base	Defines interface for getWeight(), getTypeName(), getInstructions(), requiresEvacuation() — pure virtual
DisasterReport	Concrete	Aggregates Location (composition), Reporter (composition), Disaster* (aggregation); holds priority, status; implements Rule of Five (copy/move/destructor)
Verifier	Static Utility	Scans description for fake keywords (15 words); checks reporter validity; triggers priority calculation
PriorityStrategy	Abstract (Strategy)	Interface for priority score calculation — pure virtual calculate(severity, weight, isOfficer)

Class / Component	Type	Responsibility
StandardPriority	Concrete (Strategy)	Formula: severity×10 + weight + (officer?20:5)
UrgentPriority	Concrete (Strategy)	Alternative strategy: severity×15 + weight×2 + (officer?30:10)
Volunteer	Abstract Base	Template Method pattern: deploy() declared final; defines virtual preDeployCheck() and performDeploy() hooks
MedicalVolunteer	Concrete	Inherits Volunteer; preDeployCheck() prepares medical kit; specialization (Doctor/Nurse/Surgeon)
TechVolunteer	Concrete	Inherits Volunteer; preDeployCheck() prepares field equipment; skill (Driver/Rescue/Technician)
ReliefCamp	Concrete	Tracks food/medicine stock per city; add/deduct with floor-zero clamping (never below 0)
DonationManager	Static Utility	Manages total relief fund (private static double); handles monetary donations and targeted allocations
ItemDonation	Concrete	Routes food/medicine donations to matching city camp; central warehouse fallback if no camp found
DisasterCategory	Enum	FLOOD, EARTHQUAKE, CYCLONE, LANDSLIDE, FIRE, DROUGHT, PANDEMIC
AlertObserver	Abstract (Observer)	Interface with pure virtual onAlert(area, message) for all agency observers
NDMAObserver	Concrete Observer	Prints "NDMA notified" on each alert call

17 DESIGN PATTERNS

17.Design Patterns

Design patterns are proven, reusable solutions to recurring software engineering problems. The DRCS applies five Gang of Four (GoF) patterns to keep each module independently extensible while maintaining clean boundaries between subsystems. Each pattern below is accompanied by the real Pakistan disaster scenario it was built to solve.

PATTERN 1 | FACTORY METHOD | Creational
createDisaster() + All Seven Disaster Subclasses

The Problem: 2022 Flood + Pandemic COVID Overlap

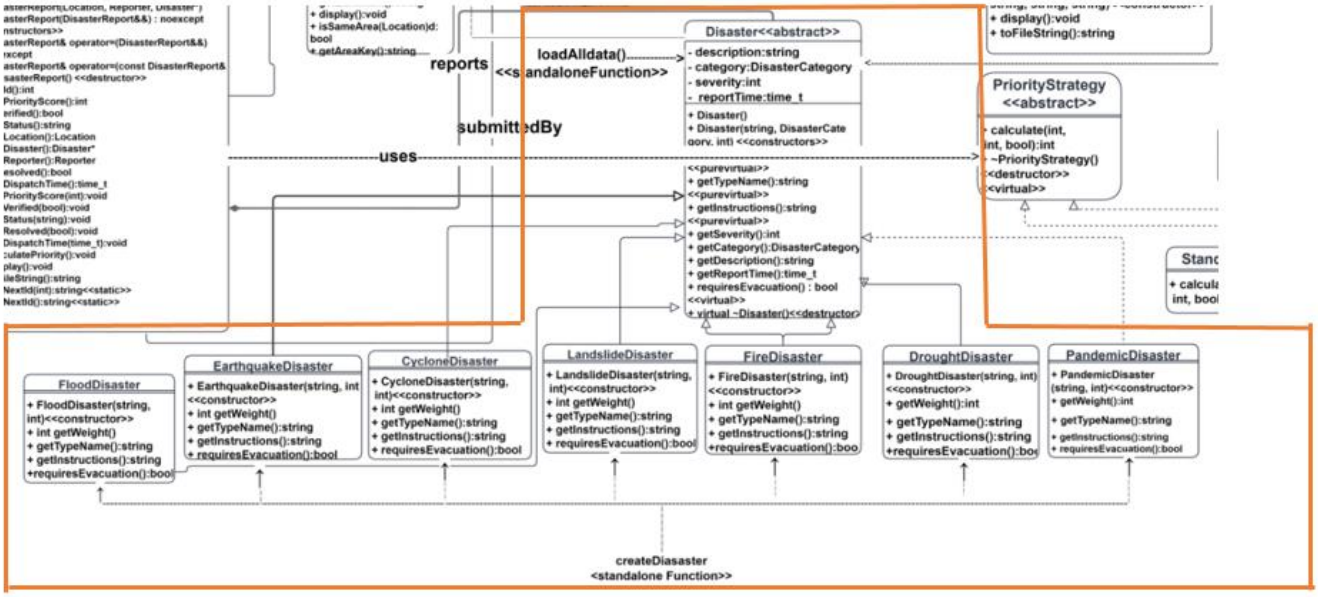
> *During the 2022 floods, Pakistan simultaneously managed active COVID protocols in relief camps. Two completely different disaster types required two completely different response protocols. A system hardcoded for one type would need to be rewritten for every new disaster.*

Intent: Define an interface for creating an object but let the factory function decide which class to instantiate, decoupling creation from usage.

Implementation: The global function createDisaster(type, desc, severity) accepts a type string, converts it to uppercase, and returns a dynamically allocated Disaster* pointing to the correct subclass. Every part of the system that needs a Disaster object calls this factory. No caller ever writes new FloodDisaster() directly.

> **Benefit:** Adding a new disaster type (e.g., Tsunami) requires only one new subclass and one new line inside createDisaster(). Zero other modules change. This is how DRCS handles Pakistan's full spectrum of disasters without code refactoring.

Disaster Type	Weight	Auto-Evacuate At	Response Instructions
FLOOD	30	Severity ≥ 6	Send boats, sandbags, and evacuation shelters
EARTHQUAKE	50	Severity ≥ 5	Send search and rescue teams and trauma kits
CYCLONE	45	Always	Order early evacuation and secure shelters
LANDSLIDE	40	Severity ≥ 7	Deploy heavy machinery and rescue teams
FIRE	35	Severity ≥ 7	Dispatch fire trucks and burn treatment units
DROUGHT	25	Never	Arrange water tankers and food supplies
PANDEMIC	55	Severity ≥ 8	Set up hospitals and enforce lockdown



PATTERN 2 | OBSERVER | Behavioral

AlertSystem + NDMAObserver + PDMAObserver + RescueObserver

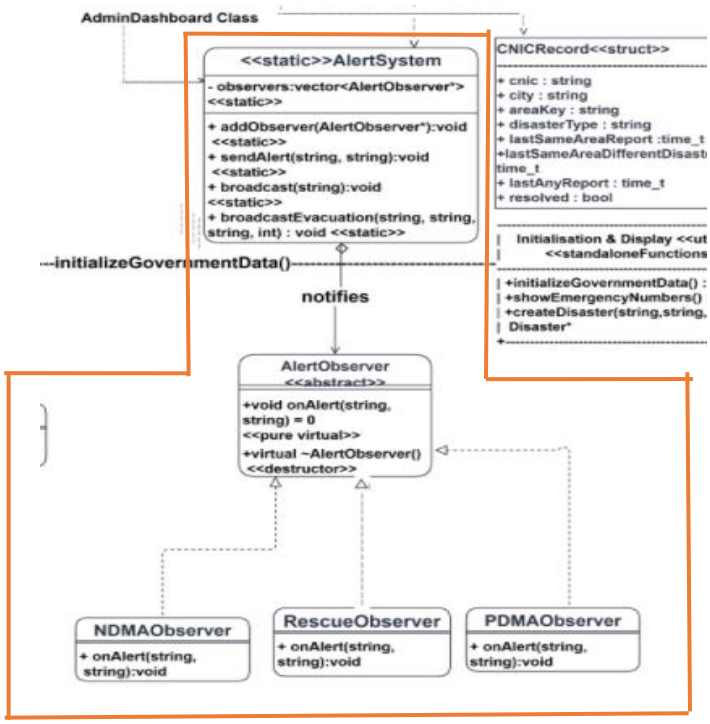
The Problem: 2005 Kashmir Earthquake — Agencies Operating in Silos

> *After the 2005 earthquake, NDMA, PDMA, and Rescue 1122 each operated their own communication channels. When a new affected area was reported, each agency had to be contacted separately — by phone. Hours passed before all three were mobilized for the same event.*

Intent: Define a one-to-many dependency so that when the subject changes state, all registered observers are notified automatically.

Implementation: AlertObserver defines a pure virtual onAlert(area, message) method. AlertSystem holds a static vector<AlertObserver*> and provides addObserver() for registration. Three concrete observers — NDMAObserver, RescueObserver, and PDMAObserver — are registered at startup. Any call to sendAlert(), broadcast(), or broadcastEvacuation() iterates this list and dispatches onAlert() to every registered agency simultaneously.

> **Benefit:** New agencies are added by creating a subclass and calling addObserver() — AlertSystem and existing observers require zero changes. Open/Closed Principle satisfied. In practice: when Cyclone Biparjoy hit in 2023, if DRCS had been deployed, all three agencies would have been notified within milliseconds of the first verified report.




```
+=================================================================+
|          REPORT SUBMITTED SUCCESSFULLY                          |
+=================================================================+
|  Report ID: 1001                                                |
|                                                                 |
|  Alert conveyed to:                                             |
|  * NDMA (National Disaster Management Authority)                |
|  * PDMA (Provincial Disaster Management Authority)              |
|  * Rescue 1122                                                  |
|  * Pakistan Army Emergency Response Unit                        |
|  * Local Administration                                         |
|                                                                 |
|  !! EVACUATION ALERT SENT TO ALL CITIZENS IN YOUR AREA !!       |
|     SMS, Radio, TV & Siren alerts activated.                    |
|                                                                 |
|  For updates, call: 1122                                        |
|  You will receive a confirmation call on your registered number. |
|  [PHONE] Please keep your phone nearby.                         |
+=================================================================+
```

```
################################################################
#          !!! EMERGENCY EVACUATION ALERT !!!                  #
#          SENT TO ALL CITIZENS IN AFFECTED AREA               #
################################################################
#                                                              #
#  [SMS]  Sent to ALL registered phone numbers in:            #
#         Karachi - Saddar
#                                                              #
#  [RADIO] Emergency broadcast activated on FM/AM channels    #
#  [TV]    Breaking news alert sent to all news channels       #
#  [SIREN] Area sirens activated                               #
#                                                              #
#  MESSAGE TO CITIZENS:                                        #
#  -------------------------------------------------------    #
#  DEAR CITIZEN OF Saddar,
#                                                              #
#  A FLOOD of SEVERITY 7/10 has been reported        #
#  in your area. IMMEDIATE EVACUATION IS REQUIRED.            #
#                                                              #
#  >> LEAVE YOUR HOMES IMMEDIATELY <<                         #
#  >> MOVE TO NEAREST RELIEF CAMP IN Karachi <<
#  >> TAKE ONLY ESSENTIAL ITEMS <<                            #
#  >> DO NOT USE ELEVATORS <<                                 #
#  >> HELP ELDERLY AND CHILDREN FIRST <<                      #
#                                                              #
#  Emergency Helpline: 1122  |  NDMA: 1131                    #
#                                                              #
################################################################
[NDMA notified] Area: Karachi - Saddar | EVACUATION ORDER ISSUED for FLOOD (Severity 7)
[Rescue 1122 notified] Area: Karachi - Saddar | EVACUATION ORDER ISSUED for FLOOD (Severity 7)
[NDMA notified] Area: Karachi - Saddar | EVACUATION ORDER ISSUED for FLOOD (Severity 7)
```

PATTERN 3 | STRATEGY | Behavioral

PriorityStrategy + StandardPriority + UrgentPriority

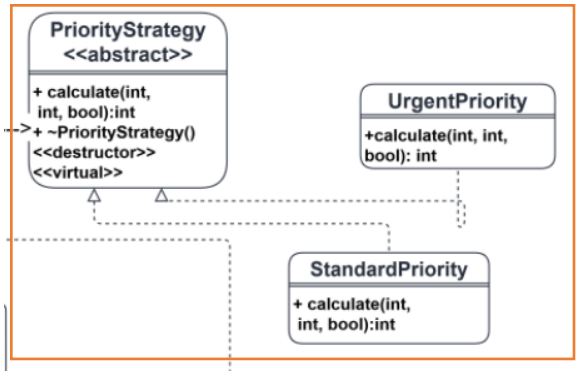
The Problem: 2010 Floods — Who Gets Help First?

> *In 2010, NDMA received thousands of simultaneous distress calls. Resources were dispatched to whichever area had the most persistent caller — not the most severe situation. A Sindh village with 10-foot floodwater waited while a dry-land administrative complaint was processed first.*

Intent: Define a family of interchangeable algorithms, encapsulate each, and make them swappable without changing the context class.

Implementation: PriorityStrategy defines a pure virtual calculate(severity, weight, isOfficer) method. StandardPriority implements the formula: severity × 10 + weight + (Officer ? 20 : 5). UrgentPriority uses severity × 15 + weight × 2 + (Officer ? 30 : 10) for high-urgency cases. DisasterReport::calculatePriority() instantiates a StandardPriority and delegates — the report class has no knowledge of the formula.

Benefit: Swapping from StandardPriority to UrgentPriority for severity ≥ 8 reports requires changing one line inside calculatePriority(). New scoring formulas can be added without modifying DisasterReport at all.




```
[VERIFIER] Report VERIFIED! Priority Score: 105
```

PATTERN 4 | TEMPLATE METHOD | Behavioral

Volunteer + MedicalVolunteer + TechVolunteer

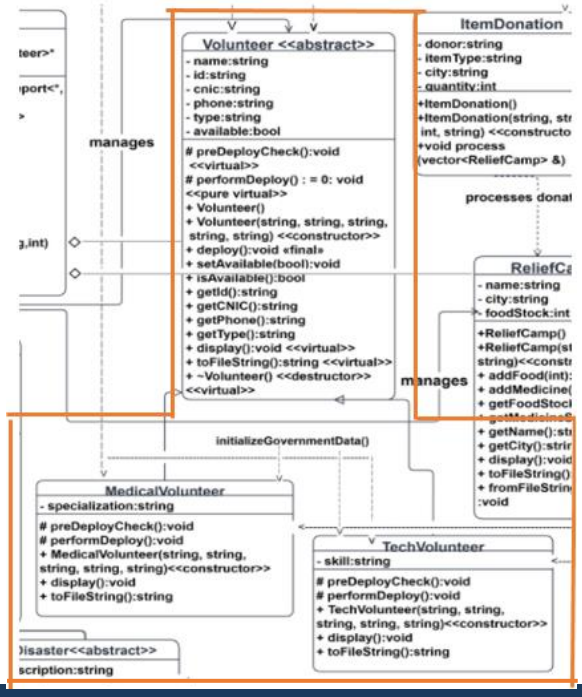
The Problem: 2022 Floods — Volunteers Showing Up Unprepared

> *Relief organizations in 2022 reported a critical gap: well-intentioned volunteers arrived at camps without the right equipment. Medical volunteers turned up without first-aid kits. Tech volunteers arrived without tools. There was no enforced pre-deployment checklist.*

Intent: Define the skeleton of an algorithm in the base class, deferring specific steps to subclasses while keeping the overall sequence fixed and inviolable.

Implementation: Volunteer::deploy() is declared final. Its sequence is: (1) guard check for availability, (2) call virtual preDeployCheck() — overridden per subclass, (3) call virtual performDeploy() — also overridden per subclass, (4) mark available = false. MedicalVolunteer prepares a medical kit and announces medical deployment. TechVolunteer prepares field equipment and announces technical deployment.

```
DEPLOYING RESOURCES for: Karachi, Saddar (Near: Empress)
   Food sent: 700 | Medicine sent: 350
Allocated Rs. 50,000 for: Emergency response. Remaining: Rs. 4,950,000
Preparing medical kit for Dr. Ahmed Khan (Doctor)...
DEPLOYING Medical Volunteer: Dr. Ahmed Khan (Doctor)
```



PATTERN 5 | FACADE | Structural

AdminDashboard + All Subsystems

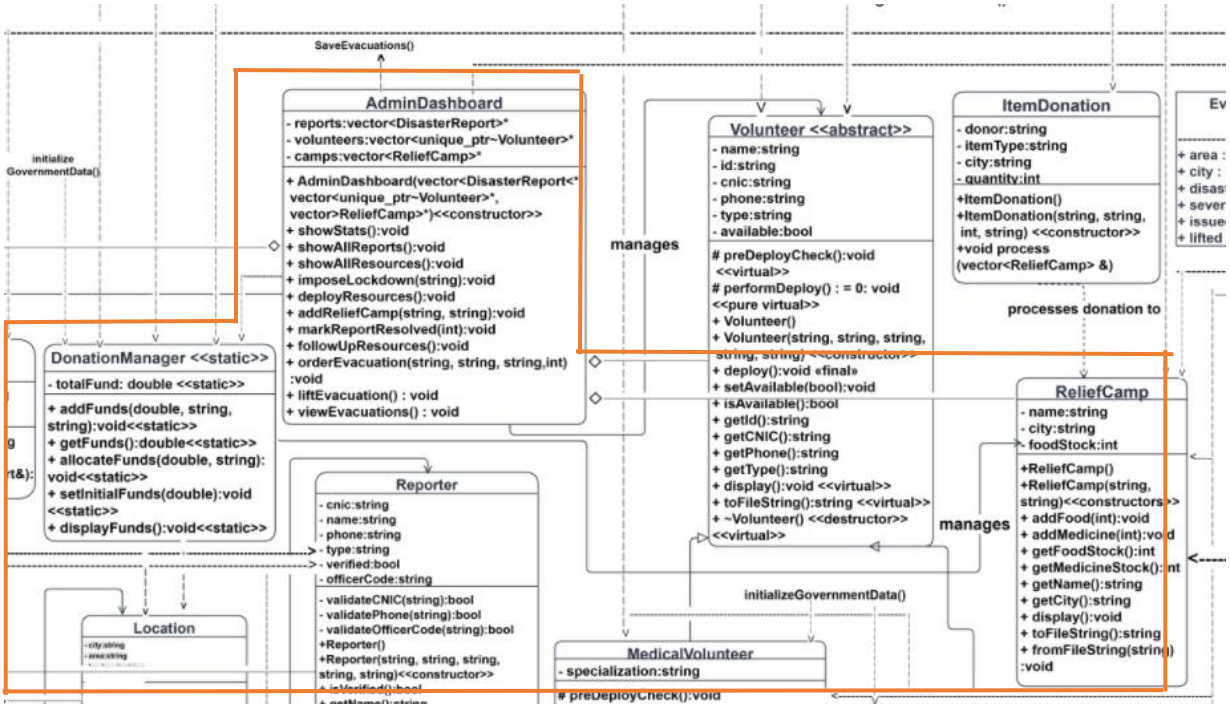
The Problem: 2022 Floods — Admin Overload, Fragmented Control

> *In the 2022 response, a single district coordinator had to simultaneously monitor WhatsApp groups, update spreadsheets, call camp managers for stock updates, liaise with NDMA by phone, and answer media queries. Administrators were the bottleneck because there was no single interface for all these operations.*

Intent: Provide a simplified, unified interface to a complex set of subsystems, hiding all internal coordination from the caller.

Implementation: AdminDashboard holds references to all primary data stores and internally coordinates multiple subsystems in each method. The main() function only calls dashboard.deployResources() or dashboard.orderEvacuation() — it never directly touches DonationManager, AlertSystem, or ReliefCamp for those operations.

Admin Method	Subsystems Coordinated Internally
deployResources()	ReliefCamp (stock deduction), DonationManager (fund allocation), Volunteer::deploy(), AlertSystem::broadcast()
imposeLockdown(area)	AlertSystem::sendAlert(), AlertSystem::broadcast(), DonationManager::allocateFunds()
orderEvacuation(...)	AlertSystem::broadcastEvacuation(), AlertSystem::sendAlert(), DonationManager::allocateFunds(), evacuation log
followUpResources()	AlertSystem::broadcast(), internal re-call of deployResources()



```
+================================+
|        ADMIN PANEL             |
+================================+
| 1.  Dashboard Stats            |
| 2.  View All Reports           |
| 3.  View Resources             |
| 4.  Impose Lockdown            |
| 5.  Order Evacuation           |
| 6.  Lift Evacuation            |
| 7.  View Evacuations Log       |
| 8.  Deploy Resources           |
| 9.  Follow Up Resources        |
| 10. Add Relief Camp            |
| 11. Mark Report Resolved       |
| 12. Save & Exit Admin          |
+================================+
Choice: 8

DEPLOYING RESOURCES for: Karachi, Gulshan e Iqbal (Near: Diamond plaza)
   Food sent: 800 | Medicine sent: 400
Allocated Rs. 50,000 for: Emergency response. Remaining: Rs. 4,950,000
Preparing medical kit for Dr. Ahmed Khan (Doctor)...
DEPLOYING Medical Volunteer: Dr. Ahmed Khan (Doctor)

[BROADCAST to ALL] Resources deployed to Karachi
[NDMA notified] Area: ALL | Resources deployed to Karachi
[Rescue 1122 notified] Area: ALL | Resources deployed to Karachi
[PDMA notified] Area: ALL | Resources deployed to Karachi

[CONFIRMATION] A confirmation message will be sent to you shortly.
```

17.6 Design Patterns Summary

#	Pattern	Category	Key Classes	Purpose
1	Factory Method	Creational	createDisaster(), Disaster subclasses	Creates correct Disaster subclass from a type string without exposing construction logic
2	Observer	Behavioral	AlertSystem, NDMAObserver, PDMAObserver, RescueObserver	Notifies all registered agencies automatically on every alert event
3	Strategy	Behavioral	PriorityStrategy, StandardPriority, UrgentPriority	Encapsulates interchangeable priority calculation algorithms
4	Template Method	Behavioral	Volunteer, MedicalVolunteer, TechVolunteer	Fixes the deployment sequence; subclasses only customize specific steps
5	Facade	Structural	AdminDashboard + all subsystems	Provides one clean interface for all admin operations, hiding subsystem complexity

18. Testing and Sample Scenarios

ID	Scenario	Input	Expected Output	Result
TC-01	Valid Flood Report (Officer)	Valid CNIC, phone, code 131, FLOOD severity 8	Verified. Priority = (8×10)+30+20 = 130. Evacuation broadcast triggered.	PASS
TC-02	Fake Report Rejection	Description contains word 'prank'	REJECTED — Fake/invalid content detected.	PASS
TC-03	CNIC 24-hour Cooldown	Same CNIC, area, disaster within 24h	BLOCKED — Help on the way. Next update in X hours.	PASS
TC-04	Different City 30-day Block	Same CNIC, different city within 30 days	BLOCKED — Next report allowed after X days.	PASS
TC-05	Resource Deployment	Admin deploys to EARTHQUAKE severity 6 report	Food: 600 units, Medicine: 300 units, Rs. 50,000 allocated, volunteer deployed.	PASS
TC-06	Item Donation — Matching City	Food, quantity 500, city Lahore	Delivered to Lahore Relief Hub. Stock += 500.	PASS
TC-07	Invalid CNIC	CNIC: 0000000000000 (leading zero)	ERROR: Invalid CNIC — Reporter not verified, report rejected.	PASS
TC-08	Auto-Evacuation — Cyclone	CYCLONE reported at any severity	Mass evacuation broadcast displayed, EvacuationRecord logged.	PASS

19. Limitations

- **Console-based UI with no graphical, web, or mobile interface** — The system runs entirely in a terminal with no GUI, web portal, or mobile app, making it inaccessible to non-technical users and unusable on smartphones in the field.
- **No real audio or voice support** — Multi-language menus exist as text only. Audio output is a single system beep and there is no voice input, speech recognition, or text-to-speech engine, leaving illiterate and injured users with no usable interface.
- **Flat-file storage with no encryption or concurrent access** — All data is saved as plain pipe-delimited .txt files. Sensitive fields including CNIC, phone, and officer codes are stored in plain text with no encryption, and the system cannot handle simultaneous access from multiple terminals.
- **No missing persons tracking or survivor triage** — There is no mechanism to log or search missing persons, nor any triage system to categorise evacuees by injury severity or flag special-needs individuals for prioritised rescue.

- **All alerts are console-simulated** — Evacuation messages, NDMA, PDMA, and Rescue 1122 notifications are all printed to screen. No actual SMS, call, or API request is ever dispatched to any real system or person.
- **No GPS or real-time location integration** — City, area, and landmark are entered manually with no validation. Incorrect or fabricated locations cannot be detected without map or coordinate verification.
- **Single hardcoded credentials with no session management** — One admin account and one universal officer code are hardcoded as plain-text constants with no RBAC, login timeout, or logout, meaning an unattended terminal grants full system access indefinitely.
- **Static fake-report detection with no audit trail** — Fake reports are caught only by a fixed keyword list trivially bypassed by rewording. There is no admin analytics dashboard or tamper-proof audit log of system actions.
- **No automated disaster signal detection** — Every report requires a human to manually type all details. There is no sensor integration, background monitoring, or auto-trigger, meaning a disaster goes unreported if no survivor is physically able to submit.

20 FUTURE IMPROVEMENTS

20.Future Improvements

- **Offline-capable mobile app with sensor-based auto-detection** — Develop a lightweight Android application that functions without continuous internet connectivity and passively monitors device sensors - accelerometer for earthquake patterns, barometer for cyclone pressure drops, microphone for flood signatures, and GPS for automatic location tagging - auto-submitting alerts without any user input and syncing data once connectivity is restored.
- **Web portal and full GUI** — Build a cross-platform web frontend using React and Django so citizens, field officers, and district admins can submit reports and manage resources from any browser without a terminal.
- **Full voice I/O in all four languages** — Integrate Whisper or Google Speech-to-Text for voice reporting and a TTS engine reading all menus and alerts aloud in Urdu, Sindhi, Pashto, and English, enabling genuinely illiterate-accessible operation.
- **Database integration with encryption and cloud sync** — Replace flat files with MySQL or PostgreSQL, encrypt sensitive fields with AES-256, hash credentials with bcrypt, and host on a cloud backend so multiple district offices share one live dataset simultaneously.
- **Missing persons registry and survivor triage** — Add a module for logging missing persons with photo, CNIC, and last known location alongside a public family-reunification search portal, and a triage system flagging evacuees by injury severity and special needs.
- **Real SMS, push notifications, and inter-agency API integration** — Replace console-print alerts with Twilio SMS and Firebase push notifications to citizens, and build a REST API layer for verified bidirectional data exchange with NDMA, PDMA, Rescue 1122, and hospital systems.
- **GPS location tagging and live disaster map** — Auto-fill location using device GPS and display all active reports, camps, and evacuation zones on a live interactive map using Leaflet.js or Google Maps API.
- **Role-based access control with multi-admin support** — Introduce permission tiers for Super Admin, District Admin, Field Officer, and Citizen with hashed credentials, session tokens, idle timeout, and two-factor authentication via OTP SMS.
- **ML-based fake report detection and analytics dashboard** — Replace the static keyword filter with an NLP classifier trained on verified reports and add an admin dashboard showing disaster frequency heatmaps, resource utilisation trends, and a tamper-proof audit log.
- **Live disaster feed integration and full regional language support** — Connect to Pakistan Meteorological Department APIs, USGS earthquake feeds, and IoT flood sensors for pre-emptive alerts, and extend translations system-wide to Balochi, Punjabi, and Saraiki alongside existing languages.

21.Conclusion

The Disaster Relief Coordination System is not just a software project — it is an answer to a question that Pakistan's disaster management infrastructure has failed to answer for decades: what does a working, unified, digital disaster coordination system look like?

The system delivers a complete pipeline — from citizen reporting and identity verification to priority-based resource deployment and mass evacuation alerts — within a single cohesive C++ application. Every feature traces back to a documented failure of Pakistan's real disaster response infrastructure:

- The Verifier and CNIC Cooldown address the unverified, duplicate reports that paralyzed NDMA hotlines in 2010.
- The PriorityStrategy address the gut-feel resource allocation that left Sindh villages waiting while administrative complaints were processed.
- The Observer pattern addresses the silo-based agency communication that delayed the 2005 Kashmir earthquake response by hours.
- The Facade pattern addresses the admin overload that made 2022 district coordinators the bottleneck of the entire relief operation.
- The multi-language system addresses the communication barrier that left Pashto and Sindhi speakers unable to navigate Urdu-only hotlines.

Five GoF design patterns structure the codebase so that each module is independently extensible without breaking others. The CNIC cooldown system and fake keyword detection show that abuse prevention can be designed into the architecture rather than bolted on afterward. File persistence ensures continuity across sessions — a lesson learned from the paper records lost in the 2005 earthquake.

While the current implementation is console-based and uses flat file storage, the architecture is deliberately designed for future extension. A database layer, a real notification API, or a GUI frontend can each be integrated by extending existing module boundaries without restructuring core logic.

The Disaster Relief Coordination System stands as both a functional disaster relief tool and a clear demonstration of how OOP principles and design patterns translate into practical, maintainable software that could — if deployed — save lives.

References

[1] B. Stroustrup, The C++ Programming Language, 4th ed. Addison-Wesley, 2013.

[2] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

[3] National Disaster Management Authority (NDMA) Pakistan. [Online]. Available: https://www.ndma.gov.pk

[4] S. Lippman, J. Lajoie, B. Moo, C++ Primer, 5th ed. Addison-Wesley, 2012.

[5] Provincial Disaster Management Authority (PDMA) Punjab. [Online]. Available: https://pdma.punjab.gov.pk

[6] United Nations Office for the Coordination of Humanitarian Affairs (OCHA). Pakistan 2010 Floods Situation Report, 2010.

[7] Earthquake Reconstruction and Rehabilitation Authority (ERRA). 2005 AJK Earthquake — Lessons Learned, 2007.

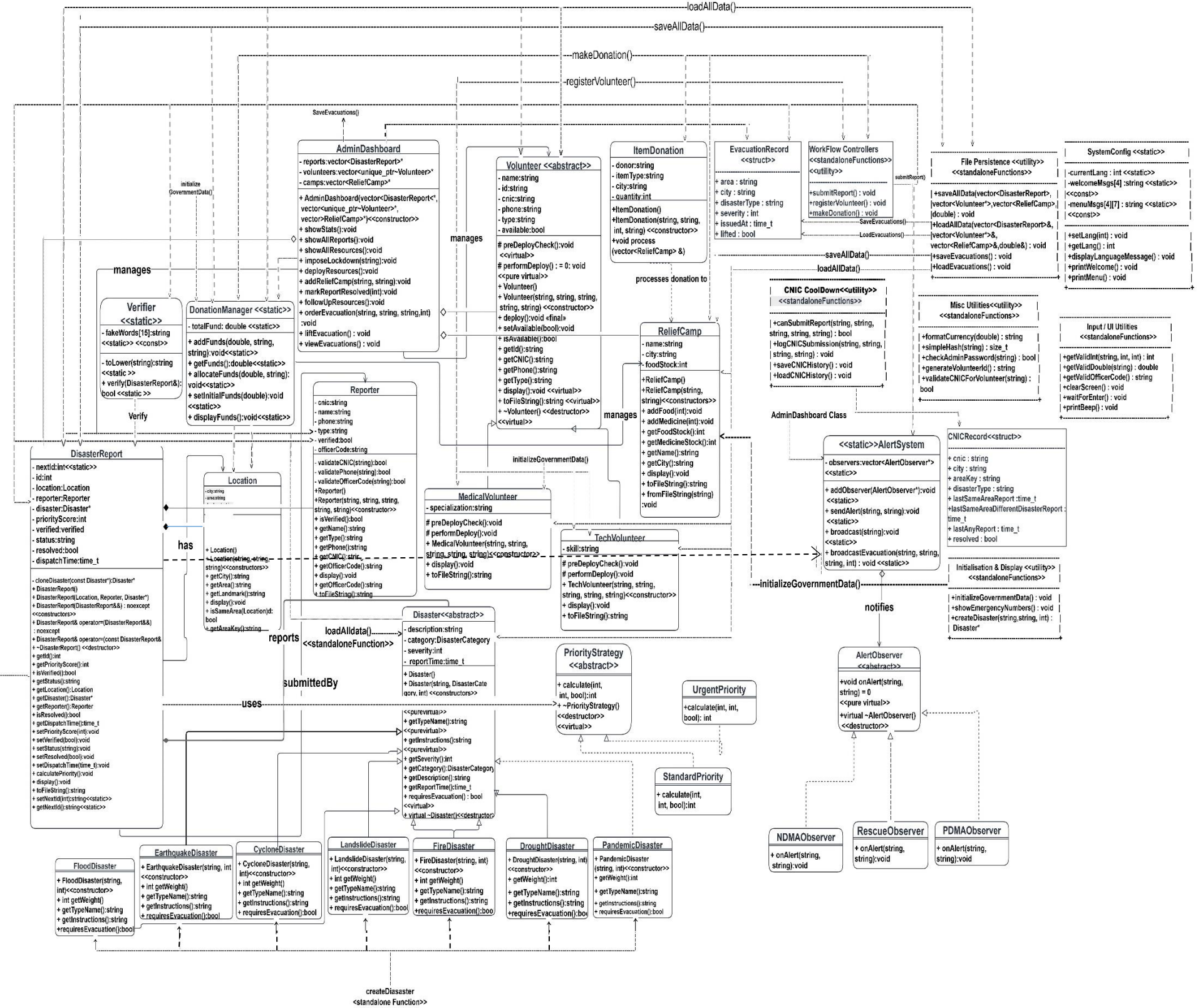
[8] PDMA Sindh. Cyclone Biparjoy Response Report, 2023.

[9] Asian Development Bank. The 2022 Pakistan Floods: Causes, Impacts, and the Way Forward, 2023.

UML Class Diagram

While free functions (standalone functions like `submitReport()`, `registerVolunteer()`, `makeDonation()`, `saveAllData()`, `loadAllData()`, and `initializeGovernmentData()` are technically not part of standard UML class diagrams, they have been explicitly included in this diagram for architectural clarity. These functions act as workflow coordinators that depend on multiple classes - their inclusion helps visualize the complete system architecture where controllers orchestrate the domain classes. In pure UML these would be modeled as "actors" or through sequence diagrams, but here they are shown as utility components to aid understanding.

UML Diagram is on the next page


loadAllData()
saveAllData()
makeDonation()
registerVolunteer()
SaveEvacuations()
initialize GovernmentData()
AdminDashboard
- reports:vector<DisasterReport>*
- volunteers:vector<unique_ptr~Volunteer>*
- camps:vector<ReliefCamp>*
+ AdminDashboard(vector<DisasterReport>*, vector<unique_ptr~Volunteer>*, vector>ReliefCamp>*)<<constructor>>
+ showStats():void
+ showAllReports():void
+ showAllResources():void
+ imposeLockdown(string):void
+ deployResources():void
+ addReliefCamp(string, string):void
+ markReportResolved(int):void
+ followUpResources():void
+ orderEvacuation(string, string, string,int) :void
+ liftEvacuation() : void
+ viewEvacuations() : void
manages
Verifier <<static>>
- fakeWords[15]:string <<static>> <<const>>
- toLower(string):string <<static >>
+ verify(DisasterReport&): bool <<static >>
Verify
DonationManager <<static>>
- totalFund: double <<static>>
+ addFunds(double, string, string):void<<static>>
+ getFunds():double<<static>>
+ allocateFunds(double, string): void<<static>>
+ setInitialFunds(double):void <<static>>
+ displayFunds():void<<static>>
Volunteer <<abstract>>
- name:string
- id:string
- cnic:string
- phone:string
- type:string
- available:bool
preDeployCheck():void <<virtual>>
performDeploy() : = 0: void <<pure virtual>>
+ Volunteer()
+ Volunteer(string, string, string, string, string) <<constructor>>
+ deploy():void «final»
+ setAvailable(bool):void
+ isAvailable():bool
+ getId():string
+ getCNIC():string
+ getPhone():string
+ getType():string
+ display():void <<virtual>>
+ toFileString():string <<virtual>>
+ ~Volunteer() <<destructor>> <<virtual>>
ItemDonation
- donor:string
- itemType:string
- city:string
- quantity:int
+ItemDonation()
+ItemDonation(string, string, int, string) <<constructor>>
+void process (vector<ReliefCamp> &)
processes donation to
EvacuationRecord <<struct>>
+ area : string
+ city : string
+ disasterType : string
+ severity : int
+ issuedAt : time_t
+ lifted : bool
WorkFlow Controllers <<standaloneFunctions>> <<utility>>
+submitReport() : void
+registerVolunteer() : void
+makeDonation() : void
submitReport()
SaveEvacuations()
LoadEvacuations()
File Persistence <<utility>> <<standaloneFunctions>>
+saveAllData(vector<DisasterReport>, vector<Volunteer*>,vector<ReliefCamp>, double) : void
+loadAllData(vector<DisasterReport>&, vector<Volunteer*>&, vector<ReliefCamp>&,double&) : void
+saveEvacuations() : void
+loadEvacuations() : void
SystemConfig <<static>>
-currentLang : int <<static>>
-welcomeMsgs[4] :string <<static>> <<const>>
-menuMsgs[4][7] : string <<static>> <<const>>
+setLang(int) : void
+getLang() : int
+displayLanguageMessage() : void
+printWelcome() : void
+printMenu() : void
CNIC CoolDown<<utility>> <<standaloneFunctions>>
+canSubmitReport(string, string, string, string, string) : bool
+logCNICSubmission(string, string, string, string) : void
+saveCNICHistory() : void
+loadCNICHistory() : void
Misc Utilities<<utility>> <<standaloneFunctions>>
+formatCurrency(double) : string
+simpleHash(string) : size_t
+checkAdminPassword(string) : bool
+generateVolunteerId() : string
+validateCNICForVolunteer(string) : bool
Input / UI Utilities <<standaloneFunctions>>
+getValidInt(string, int, int) : int
+getValidDouble(string) : double
+getValidOfficerCode() : string
+clearScreen() : void
+waitForEnter() : void
+printBeep() : void
ReliefCamp
- name:string
- city:string
- foodStock:int
+ReliefCamp()
+ReliefCamp(string, string)<<constructors>>
+ addFood(int):void
+ addMedicine(int):void
+ getFoodStock():int
+ getMedicineStock():int
+ getName():string
+ getCity():string
+ display():void
+ toFileString():string
+ fromFileString(string) :void
AdminDashboard Class
<<static>>AlertSystem
- observers:vector<AlertObserver*> <<static>>
+ addObserver(AlertObserver*):void <<static>>
+ sendAlert(string, string):void <<static>>
+ broadcast(string):void <<static>>
+ broadcastEvacuation(string, string, string, int) : void <<static>>
CNICRecord<<struct>>
+ cnic : string
+ city : string
+ areaKey : string
+ disasterType : string
+ lastSameAreaReport :time_t
+lastSameAreaDifferentDisasterReport : time_t
+ lastAnyReport : time_t
+ resolved : bool
Initialisation & Display <<utility>> <<standaloneFunctions>>
+initializeGovernmentData() : void
+showEmergencyNumbers() : void
+createDisaster(string,string, int) : Disaster*
Reporter
- cnic:string
- name:string
- phone:string
- type:string
- verified:bool
- officerCode:string
- validateCNIC(string):bool
- validatePhone(string):bool
- validateOfficerCode(string):bool
+Reporter()
+Reporter(string, string, string, string, string)<<constructor>>
+ isVerified():bool
+ getName():string
+ getType():string
+ getPhone(:string
+ getCNIC():string
+ getOfficerCode():string
+ display():void
+ getOfficerCode():string
+ toFileString():string
initializeGovernmentData()
MedicalVolunteer
- specialization:string
preDeployCheck():void
performDeploy():void
+ MedicalVolunteer(string, string, string, string, string)<<constructor>>
+ display():void
+ toFileString():string
TechVolunteer
- skill:string
preDeployCheck():void
performDeploy():void
+ TechVolunteer(string, string, string, string, string)<<constructor>>
+ display():void
+ toFileString():string
DisasterReport
- nextId:int<<static>>
- id:int
- location:Location
- reporter:Reporter
- disaster:Disaster*
- priorityScore:int
- verified:verified
- status:string
- resolved:bool
- dispatchTime:time_t
- cloneDisaster(const Disaster*):Disaster*
+ DisasterReport()
+ DisasterReport(Location, Reporter, Disaster*)
+ DisasterReport(DisasterReport&&) : noexcept <<constructors>>
+ DisasterReport& operator=(DisasterReport&&) : noexcept
+ DisasterReport& operator=(const DisasterReport&
+ ~DisasterReport() <<destructor>>
+ getId():int
+ getPriorityScore():int
+ isVerified():bool
+ getStatus():string
+ getLocation():Location
+ getDisaster():Disaster*
+ getReporter():Reporter
+ isResolved():bool
+ getDispatchTime(:time_t
+ setPriorityScore(int):void
+ setVerified(bool):void
+ setStatus(string):void
+ setResolved(bool):void
+ setDispatchTime(time_t):void
+ calculatePriority():void
+ display():void
+ toFileString():string
+ setNextId(int):string<<static>>
+ getNextId():string<<static>>
has
Location
- city:string
- area:string
+ Location()
+ Location(string, string, string)<<constructors>>
+ getCity():string
+ getArea():string
+ getLandmark():string
+ display():void
+ isSameArea(Location)d: bool
+ getAreaKey():string
reports
loadAlldata() <<standaloneFunction>>
Disaster<<abstract>>
- description:string
- category:DisasterCategory
- severity:int
- reportTime:time_t
+ Disaster()
+ Disaster(string, DisasterCategory, int) <<constructors>>
<<purevirtual>>
+ getTypeName():string
<<purevirtual>>
+ getInstructions():string
<<purevirtual>>
+ getSeverity():int
+ getCategory():DisasterCategory
+ getDescription():string
+ getReportTime():time_t
+ requiresEvacuation() : bool <<virtual>>
+ virtual ~Disaster()<<destructor>
submittedBy
uses
initializeGovernmentData()
notifies
PriorityStrategy <<abstract>>
+ calculate(int, int, bool):int
+ ~PriorityStrategy() <<destructor>> <<virtual>>
UrgentPriority
+calculate(int, int, bool): int
StandardPriority
+ calculate(int, int, bool):int
AlertObserver <<abstract>>
+void onAlert(string, string) = 0 <<pure virtual>>
+virtual ~AlertObserver() <<destructor>>
NDMAObserver
+ onAlert(string, string):void
RescueObserver
+ onAlert(string, string):void
PDMAObserver
+ onAlert(string, string):void
createDisaster <standalone Function>>
FloodDisaster
+ FloodDisaster(string, int)<<constructor>>
+ int getWeight()
+ getTypeName():string
+ getInstructions():string
+requiresEvacuation():bool
EarthquakeDisaster
+ EarthquakeDisaster(string, int <<constructor>>
+ int getWeight()
+ getTypeName():string
+ getInstructions():string
+ requiresEvacuation():bool
CycloneDisaster
+ CycloneDisaster(string, int)<<constructor>>
+ int getWeight()
+ getTypeName():string
+ getInstructions():string
+ requiresEvacuation():bool
LandslideDisaster
+ LandslideDisaster(string, int)<<constructor>>
+ int getWeight()
+ getTypeName():string
+ getInstructions():string
+ requiresEvacuation():bool
FireDisaster
+ FireDisaster(string, int) <<constructor>>
+ int getWeight()
+ getTypeName():string
+ getInstructions():string
+requiresEvacuation():boo
DroughtDisaster
+ DroughtDisaster(string, int) <<constructor>>
+ getWeight():int
+ getTypeName():string
+ getInstructions():string
+requiresEvacuation():bo
PandemicDisaster
+ PandemicDisaster (string, int)<<constructor>>
+ getWeight():int
+ getTypeName():string
+ getInstructions():string
+ requiresEvacuation():bool
createDisaster <standalone Function>>